HLT Tracker Reconstruction

Marco Musich[1]
on behalf of TSG and NGT

1. CERN EP-CMS-TDQ

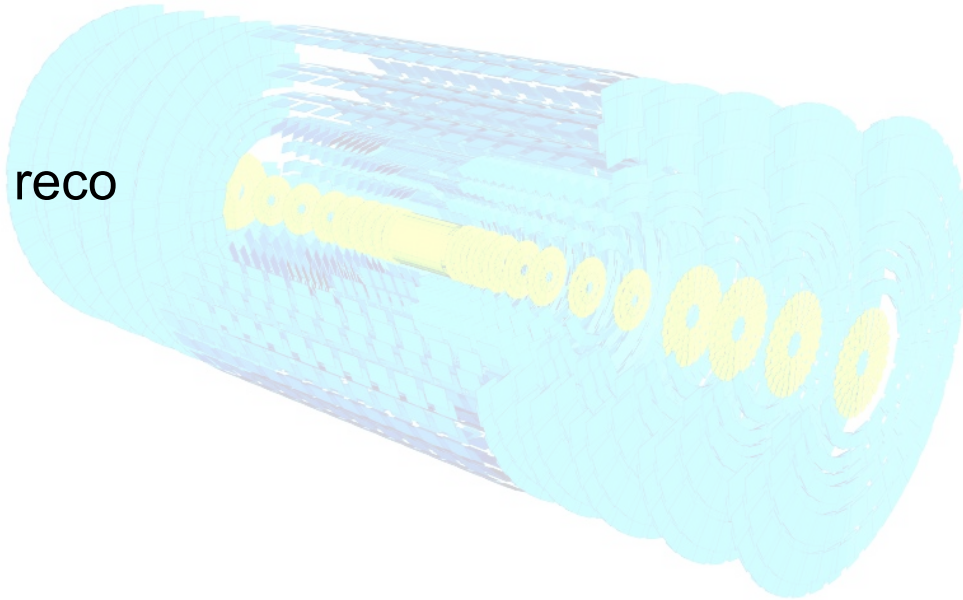
CMS Tracker DPG Workshop: *ULB/VUB Bruxelles*
October 15th, 2026

NexTGen
Next Generation Triggers

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

Outline of the talk

- **The HLT menu …**
  - … and its incarnation for Run 4
- **Available Phase-2 HLT menus**
- **Available Phase-2 HLT paths**
- **Importance of RAW data format for HLT studies**
- **HLT Local Reconstruction in general**
  - Discussion on Heterogeneous IT local reco
  - Discussion on options for Heterogeneous OT local reco
- **Heterogeneous Vector Hits**
- **Conclusions & Outlook**



Very short intro to the HLT (High Level Trigger)

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **The HLT selects which events accepted by the L1-Trigger are kept for permanent storage and offline processing (also, which ones are used for online DQM).**

- **To make this decision, the HLT runs its own version of the event reconstruction (usually faster than the offline reconstruction) on a dedicated computing cluster at P5 (HLT farm).**

- **The input to HLT are events in RAW-data format (we don't have that yet fully available for Phase 2 setup), assembled by the DAQ system.**

- **These events are reconstructed on HLT nodes running an "HLT menu" with CMSSW:**
  - a "HLT menu" is - ultimately - a CMSSW configuration file which defines a collection of triggers paths;
  - events not selected by any of these triggers paths are lost, i.e. not kept for offline processing;

The High Level Trigger Menu (in Run 3)

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

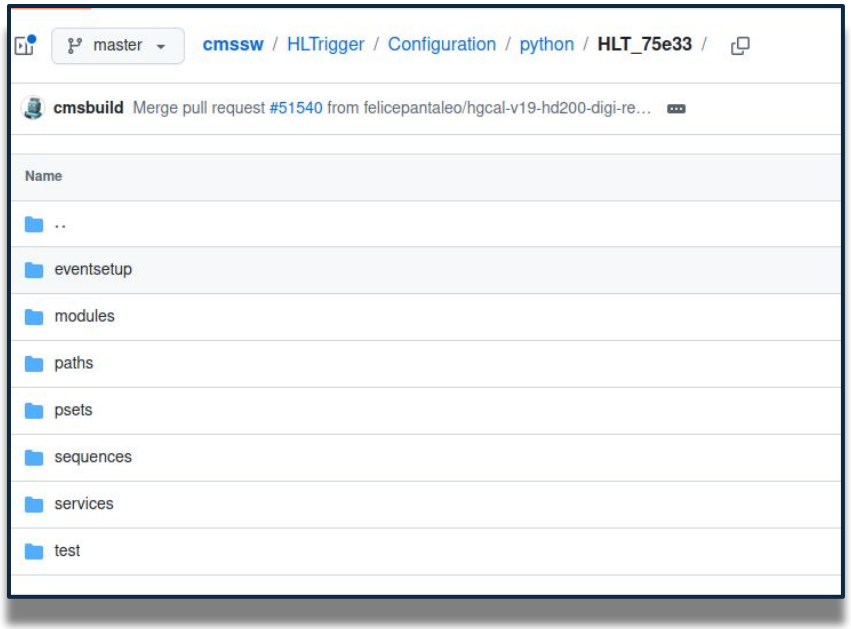
- **The HLT menu in Run 3 was a huge cmssw configuration python file (several hundreds of thousands lines of code) running O(10^3) HLT paths**
  - Example: link
  - Each path is defined by a sequence of reconstruction (`EDProducer`s) and filtering (`EDFilter`s) modules;
- **`ConfDB` is (*was*) the tool we use in CMS to store / modify / deploy HLT menus for data-taking and offline development.**
  - There is more information about this model in the TSG tutorials of the past years (indico)



The High Level Trigger Menu (for Run 4)

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- Since shortly after the time of the HLT TDR (~ Spring 2022) a "Simplified HLT Menu" from the Phase-2 DAQ-HLT-TDR lives directly into `cmssw` (original PR in CMSSW_12_4_X)
  - At the moment there is no direct counterpart in `confDB` (yes, but replacement s coming, see slides).
- At the time it was decided to arrange the code in subfolders within the directory HLTrigger/Configuration/python/HLT_75e33/ split following a similar approach to what is used to organize the configuration in `confDB` (eventsetup, modules, sequences, paths, etc.)



- A consequence of this arrangement is that (at variance with respect to Run 3) trigger contacts must develop directly `cmssw` python configuration and create Pull Requests against the master branch of the official cms-sw fork.
- The review is carried out directly by @cms-sw/hlt-l2 (i.e. STORM) and *NOT* via CMSHLT JIRA tickets (with its pros and cons)
- Again, differently from Run 3, the HLT configuration in this area is NOT self-contained (for example the list of products to be saved by an HLT job is not defined within the menu itself, but needs to be configured elsewhere)

Available “Phase 2” HLT Menus

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **Over the course of the years a few different menus have been introduced in `cmssw` to facilitate different tasks:**
  - **HLTrigger/Configuration/python/HLT_75e33_cff.py** : the **main menu**, including “MC” paths (without filtering, only reconstruction) to help validate / study HLT objects reconstructions as well as E/γ unseeded paths;
  - **HLTrigger/Configuration/python/HLT_75e33_timing_cff.py** : menu used to run “realistic” timing and resource consumption benchmarks, equivalent to the main menu, stripped of the “MC” and the E/γ paths;
  - **HLTrigger/Configuration/python/HLT_NGTScouting_cff.py** : dedicated menu for NGT studies, runs **filterless** HLT reconstruction over the whole set of the Global L1 accept.
  - **HLTrigger/Configuration/python/HLT_75e33_trackingOnly_cff.py** : specialized menu dedicated just to track / vertex reconstruction, runs only **one** MC path for tracking.
  - **HLTrigger/Configuration/python/HLT_HeterogeneousOnly_cff.py** : specialized menu dedicated to study just the heterogeneous part of the reconstruction, obtained stripping the main menu of everything other than the “portable” `alpaka` modules and the dependencies needed in order to run (conversions, eventsetup, etc.) .
- **What menus are NOT available (yet):**
  - Heavy Ions (and other kind of beam species)- related configuration
  - Special configurations (e.g. cosmics, etc.)

Structure of a typical Trigger in the HLT

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **In CMSSW terms, a trigger is implemented as a `cms.Path` made of producers and filters;**
  - **Important**: intermediate filters are used to reject uninteresting events as early as possible (the decision of a trigger is the logical AND of its filters);
  - **if an intermediate filter rejects an event, later steps of that trigger do not run for that event**
    - more complex, time-consuming, parts of the reconstruction run only when necessary

- **Important: every trigger `Path` must be self-contained**
  - every `Path` must be able to run standalone, without relying on the existence of other `Path`s (a few exceptions to this rule exist).

- **Any given module/sequence can be used by multiple Paths:**
  - **good to reuse existing modules/sequences whenever possible**
  - adding new modules **increases** timing of HLT jobs;

Structure of a typical Trigger in the HLT

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **Typical structure of physics triggers:**
  - Name: **`HLT_*`**
  - Starts with **`HLTBeginSequence`** (**`gitHub`**), which will eventually contain selection on event type (e.g. Physics, Calibration, Random) and basic infrastructure
  - **L1T "seed"**: filter on L1T algorithms
  - (**not yet implemented in Phase-2**) **HLT "prescaler"**: filter to apply HLT prescale;
  - **Reconstruction** (sequences/modules) interleaved with filters;
  - Ends with **`HLTEndSequence`** (**`gitHub`**)

- **In general, HLT menus follow (many) long-standing naming conventions, for example:**
  - producers and filters are called **`hlt*`** w/o underscores (e.g. **`hltOnlineBeamSpot`**)
  - Sequences are called **`HLT*`** w/o underscores (e.g. **`HLTTrackingSequence`**)

Only a few tens of paths, and not a physics menu

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

Path name	75e33	75e33_timing	NGTScouting	trackingOnly	heterogeneousOnly
DQM_HGCALHeterogeneousValidation	Y	N	N	N	N
DQM_TRKHeterogeneousValidation	Y	N	N	N	N
DST_HeterogeneousReco	N	N	N	N	Y
DST_NGTScouting	N	N	Y	N	N
DST_PFScouting	Y	N	N	N	N
HLT_AK4PFPuppiJet520	Y	Y	N	N	N
HLT_Diphoton30_23_IsoCaloId_L1Seeded	Y	Y	N	N	N
HLT_Diphoton30_23_IsoCaloId_Unseeded	Y	N	N	N	N
HLT_DoubleEle23_12_Iso_L1Seeded	Y	Y	N	N	N
HLT_DoubleEle25_CaloIdL_PMS2_L1Seeded	Y	N	N	N	N
HLT_DoubleEle25_CaloIdL_PMS2_Unseeded	Y	Y	N	N	N
HLT_DoubleMediumChargedIsoPFTauHPS40_eta2p1	Y	Y	N	N	N
HLT_DoubleMediumDeepTauPFTauHPS35_eta2p1	Y	Y	N	N	N
HLT_DoublePFPuppiJets128_DoublePFPuppiBTagDeepCSV_2p4	Y	Y	N	N	N
HLT_DoublePFPuppiJets128_DoublePFPuppiBTagDeepFlavour_2p4	Y	Y	N	N	N
HLT_Ele115_NonIso_L1Seeded	Y	Y	N	N	N
HLT_Ele26_WP70_L1Seeded	Y	Y	N	N	N
HLT_Ele26_WP70_Unseeded	Y	N	N	N	N
HLT_Ele30_WPTight_L1Seeded_LooseDeepTauPFTauHPS30_eta2p1_CrossL1	Y	Y	N	N	N
HLT_Ele32_WPTight_L1Seeded	Y	Y	N	N	N
HLT_Ele32_WPTight_Unseeded	Y	N	N	N	N
HLT_IsoMu20_eta2p1_LooseDeepTauPFTauHPS27_eta2p1_CrossL1	Y	Y	N	N	N
HLT_IsoMu24_FromL1TkMuon	Y	Y	N	N	N
HLT_LooseDeepTauPFTauHPS150_L2NN_eta2p1	Y	Y	N	N	N
HLT_Mu17_TrkIsoVVL_Mu8_TrkIsoVVL_DZ_FromL1TkMuon	Y	Y	N	N	N
HLT_Mu37_Mu27_FromL1TkMuon	Y	Y	N	N	N
HLT_Mu50_FromL1TkMuon	Y	Y	N	N	N
HLT_PFHT200PT30_QuadPFPuppiJet_70_40_30_30_TriplePFPuppiBTagDeepFlavour_2p4	Y	Y	N	N	N
HLT_PFHT330PT30_QuadPFPuppiJet_75_60_45_40_TriplePFPuppiBTagDeepFlavour_2p4	Y	Y	N	N	N
HLT_PFPuppiHT1070	Y	Y	N	N	N
HLT_PFPuppiMETTypeOne140_PFPuppiMHT140	Y	Y	N	N	N
HLT_Photon108EB_TightID_TightIso_L1Seeded	Y	Y	N	N	N
HLT_Photon108EB_TightID_TightIso_Unseeded	Y	N	N	N	N
HLT_Photon187_L1Seeded	Y	Y	N	N	N
HLT_Photon187_Unseeded	Y	N	N	N	N
HLT_TriMu_10_5_5_DZ_FromL1TkMuon	Y	Y	N	N	N
MC_BTV	Y	N	N	N	N
MC_Ele5_Open_L1Seeded	Y	N	N	N	N
MC_Ele5_Open_Unseeded	Y	N	N	N	N
MC_JME	Y	N	N	N	N
MC_TRK	Y	N	N	Y	N

- **A (more readable) digest of what is available in terms of paths (and a short description of the main modules used in them) is available here.**
  - including instructions on how to produce a more detailed dump of the menu
- **Generally speaking only a few tens of paths (including service paths, meant for DQM / Validation) are available.**
- **The "Simplified Menu" from the TDR mostly contains paths needed to exercise a variety of objects reconstruction and is NOT suitable for a realistic physics program (help needed from PAGs!).**
  - See dedicated discussion at the last CMS week

About half of the L1T seeds are unused

Seed	Usage
pDoubleEGEle37_24	USED
pDoubleIsoTkPho22_12	USED
pDoublePuppiJet112_112	USED
pDoublePuppiJet160_35_mass620	UNUSED
pDoublePuppiTau52_52	USED
pDoubleTkEle25_12	USED
pDoubleTkElePuppiHT_8_8_390	UNUSED
pDoubleTkMuPuppiHT_3_3_300	UNUSED
pDoubleTkMuPuppiJetPuppiMet_3_3_60_130	UNUSED
pDoubleTkMuon15_7	USED
pDoubleTkMuonTkEle5_5_9	UNUSED
pDoubleTkMuon_4_4_OS_Dr1p2	UNUSED
pDoubleTkMuon_4p5_4p5_OS_Er2_Mass7to18	UNUSED
pDoubleTkMuon_OS_Er1p5_Dr1p4	UNUSED
pIsoTkEleEGEle22_12	USED
pNNPuppiTauPuppiMet_55_190	UNUSED
pPuppiHT400	USED
pPuppiHT450	USED
pPuppiMET200	USED
pPuppiMHT140	UNUSED
pPuppiTauTkIsoEle45_22	USED
pPuppiTauTkMuon42_18	USED
pQuadJet70_55_40_40	USED
pSingleEGEle51	USED
pSingleIsoTkEle28	USED
pSingleIsoTkPho36	USED
pSinglePuppiJet230	USED
pSingleTkEle36	USED
pSingleTkMuon22	USED
pTkEleIsoPuppiHT_26_190	UNUSED
pTkElePuppiJet_28_40_MinDR	UNUSED
pTkEleTkMuon10_20	UNUSED
pTkMuPuppiJetPuppiMet_3_110_120	UNUSED
pTkMuTriPuppiJet_12_40_dRMax_DoubleJet_dEtaMax	UNUSED
pTkMuonDoubleTkEle6_17_17	UNUSED
pTkMuonPuppiHT6_320	UNUSED
pTkMuonTkEle7_23	UNUSED
pTkMuonTkIsoEle7_20	UNUSED
pTripleTkMuon5_3_3	USED
pTripleTkMuon_5_3_0_DoubleTkMuon_5_3_OS_MassTo9	UNUSED
pTripleTkMuon_5_3p5_2p5_OS_Mass5to17	UNUSED

- **Seeds available in the “step1” L1T Phase 2 menu are underutilized:**
  - Only around 50% are effectively used to seed HLT paths in the current “Simplified menu”;
- **Not all the paths in the HLT menu use the corresponding L1T objects participating to the L1T decision to seed the HLT reconstruction:**
  - In the Run-3 case this is true for the vast majority of the paths;
  - In the Phase-2 case many path run an “unseeded” reconstruction (which is good for scouting purposes, but not efficient timing-wise)
- **Recently migrated all paths to consume P2GT inputs:**
  - #51669, #51720 , #51881, #51927
- **Necessary to tighten the communication and collaboration between the HLT and L1T Menu experts (as well as P2GT experts).**

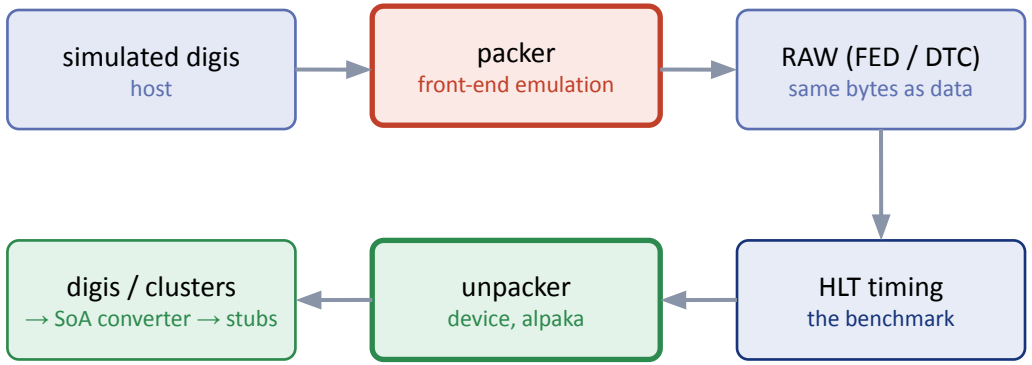
Importance of the RAW DataFormat

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

An HLT benchmark is only as realistic as its input: the trigger starts from FED/DTC buffers, not from simulated digis

- **Why a realistic RAW format matters for the benchmark:**
  - **Unpacking is part of the budget**. Today's Phase-2 HLT workflows start from simulated digis or clusters and skip RAW→digi entirely; in Run 3 the pixel unpacker is one of the heavier device kernels. Timing without it is optimistic.
  - **Volume drives the architecture**. At ~750 kHz L1 rate and several MB per event, the RAW payload sets the H2D copies, the device memory footprint and the per-event latency, none of which a digi-based input exercises.
  - **The format decides what the HLT can see**. Front-end encoding, bit widths and truncations fix which quantities exist, at what precision, and where in the chain.
- **Why a packer / unpacker cycle, for both IT and OT:**
  - **Packer (digi → RAW)** in simulation: emulate the front-end and DTC output so MC and data enter the HLT through the same door.
  - **Unpacker (RAW → digi)**, CPU reference first, then on device with the same pattern as the Run-3 SiPixelRawToCluster; the OT one hands the stubs/clusters straight to the SoA converter.

The cycle, one per sub-detector (IT and OT)



compare digi ↔ digi: lossless where the spec says so

From the Computing CDR schedule, first version due EOY2026

2026	H2	Phase 2 MC with agreed geometry used for CSA28 preparation and validated (first version of) packer/unpacker.	Initiate infrastructure upgrade
		Run 3 reprocessing (if any)	
		LS 3 starts	
	H1	GPU workflows in RelVal (larger offload fraction)	Finalize the plan for infrastructure update including Tier-0, DMWM, CRAB, Analysis facilities
		Production-ready RNTuple Source and OutputModule	
		2026 L1T and TSG MC Production	Data taking and Run 3 MC production; ARM as opportunistic resources
		Release for 2026 data taking	

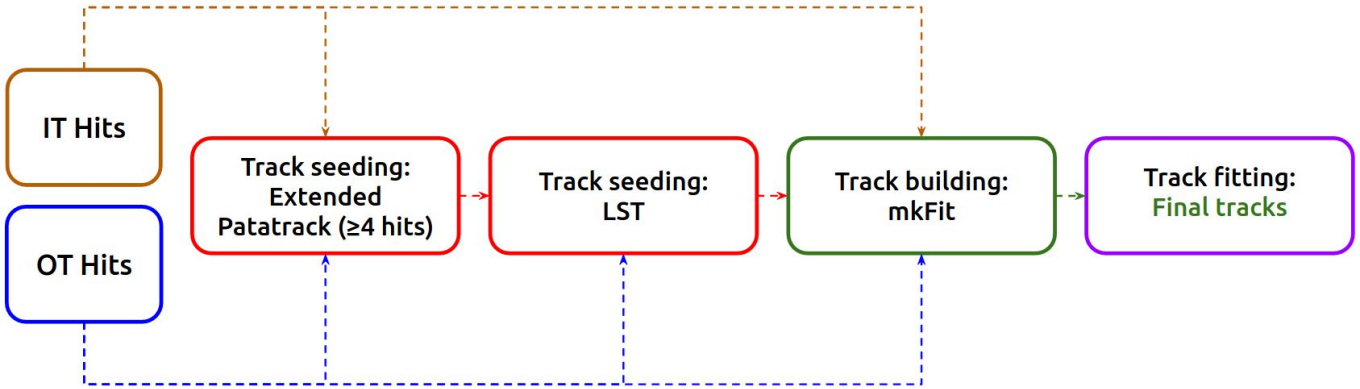
Current HLT Tracking Baseline

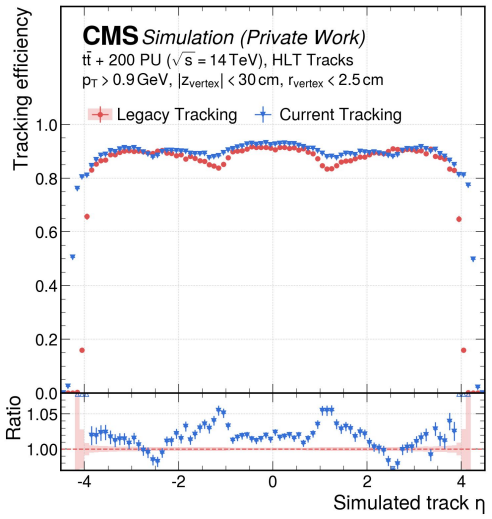
Tracking components of the current reconstruction baseline:

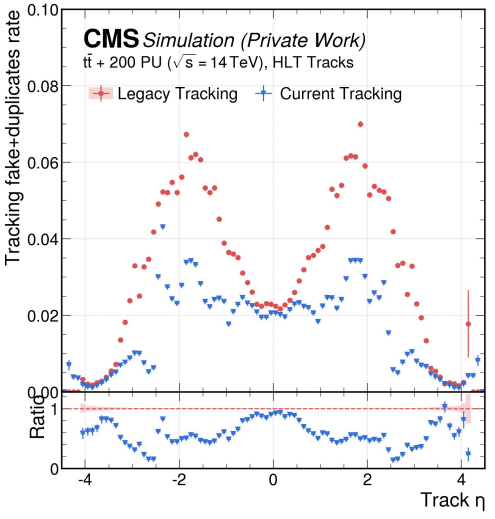
1. **CA (aka "Patatrack"):** produces Seed Tracks (including fit) using IT + 3 inner layers of OT (PS modules) → used as prompt seeds (**enables a single full scan tracking iteration!**)
2. **LST**: extension of PixelTracks in OT + OT-only, displaced seeds
3. **mkFit**: track building algorithm in full tracker
4. **CKF**: track fitting algorithm
5. **Track-purity DNN** at multiple steps.

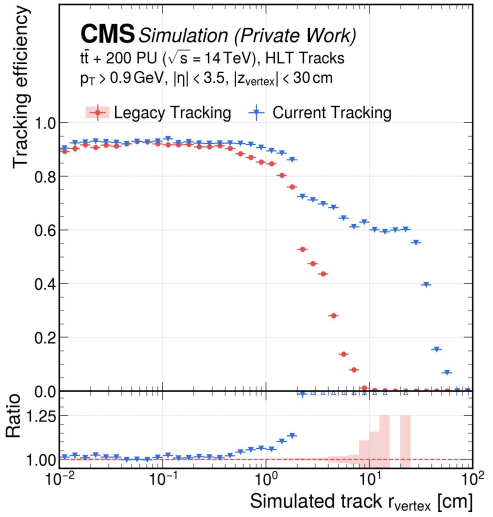
From **Legacy baseline** to the **current Tracking** achieves

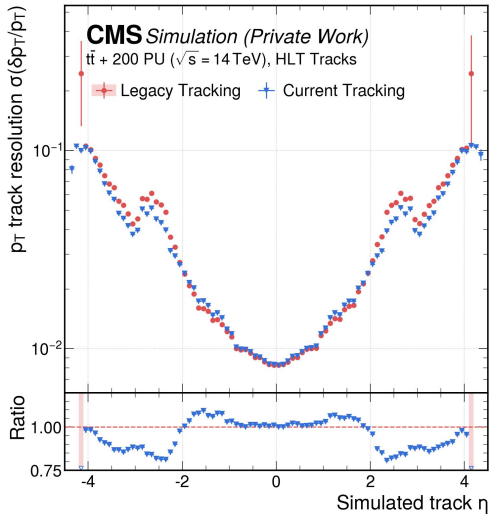
- Overall **higher efficiency,** at **lower fake and duplicate rate**
- With comparable track parameter resolution
- While being **5 times faster when running on GPUs.**











HLT Tracker Local Reconstruction

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

From digis to rechits for the Inner and Outer Tracker in the Phase-2 HLT menu (HLT_75e33), and what it costs

The sequences:

Inner Tracker (pixel) — portable, on device

`hltSiPixelClustersSoA` : digi → cluster directly from the simulated digis (no RAW unpacker yet, see previous slide).
`hltSiPixelRecHitsSoA` : CPE on device, SoA output consumed by the pixel CA.
`hltSiPixelClusters` / `hltSiPixelRecHits` : SoA → legacy conversion for the host consumers.
`hltSiPixelClusterShapeCache` : host; cluster-shape filter cache for the iterative steps.

Outer Tracker (PS / 2S) — still legacy, on host

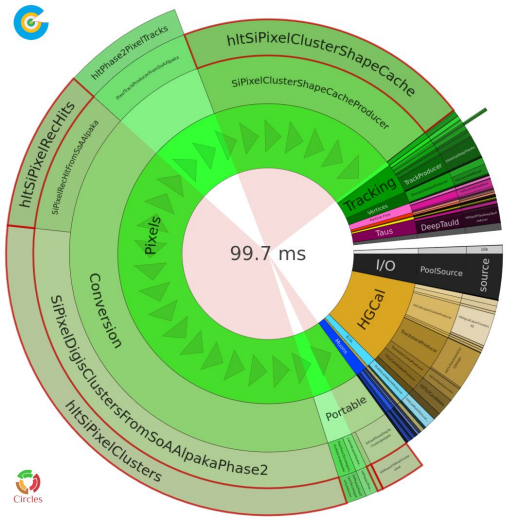
`hltSiPhase2Clusters` : `Phase2TrackerClusterizer` on the OT digis.
`hltSiPhase2RecHits` : `Phase2TrackerRecHits` , input to MkFit / CKF and to the new `OTRecHitsSoA` converter feeding the GPU VectorHits former.

Which paths run them:

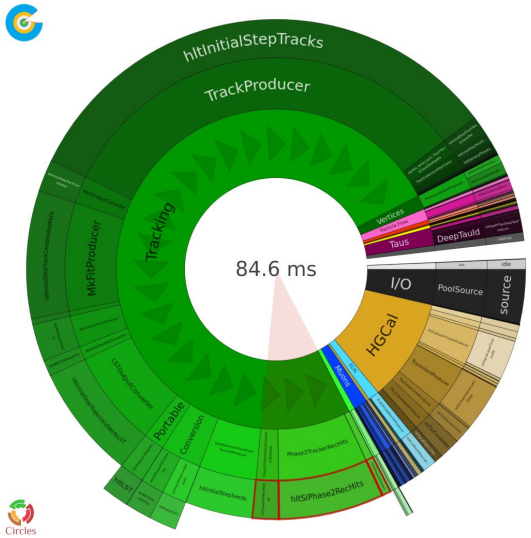
Pixel local reco sits at the head of `HLTPhase2PixelTracksAndVerticesSequence`, i.e. in every path that needs pixel tracks or vertices: muons, e/γ pixel matching, taus, PF-based jets/MET/HT, b-tagging: effectively the whole menu once an L1 seed fires.

OT local reco runs in the full tracking sequence (`HLTTrackingSequence`) used by the PF and isolation paths, and now also by the stub-aware pixel CA.

Timing: `FastTimerService`, Circles view



Pixels group highlighted
99.7 ms / event total



Tracking group highlighted
84.6 ms / event total

IT Local Reconstruction

- **The HLT runs the portable (Patatrack-style) chain, offline runs the legacy one**

Step	Offline (legacy)	HLT (portable, alpaka)	Differences that matter
Input	digis (RAW unpacker in Run 3; Phase-2 from simulated digis)	simulated digis, no RAW yet	identical today; diverges once a Phase-2 unpacker exists
Clustering	`PixelThresholdClusterizer` on host, per-pixel and per-cluster charge thresholds	`SiPixelPhase2DigiToCluster@alpaka`: device connected-component, per-module caps on pixels and clusters	truncation at the caps, cluster-charge cut, handling of edge/big pixels
CPE	`PixelCPEGeneric` (Lorentz drift, edge handling, position-dependent errors)	`PixelCPEFastParams`: simplified, parametrised on device	position bias, error model → seeding windows and χ^2 in the fit
Output	`SiPixelCluster / SiPixelRecHit` collections	SoA; converted back to legacy for host consumers	conversion cost; two objects to keep in sync
Precision	`double`	`float`	borderline clusters/hits flip at cuts

- **Who owns the code?**
  - **DPG owns the physics**: CPE parameters, charge thresholds, Lorentz angle, bad-pixel handling
  - **NGT / HLT owned** so far the implementation: kernels, SoA formats, timing, portability across backends.
  - **Why this matters now:** a recent bugfix in the portable chain came from NGT (cms-sw PR #52174). The fix was found while chasing performance, not by a validation that compared the two chains.
    - We need better validation / cross-checks of the physics performance also on the DPG side of the portable code.

OT Local Reconstruction

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

Current: Phase-2 OT extension

PS P-hits of the OT barrel as extra pixel layers

hltSiPhase2Clusters → hltSiPhase2RecHits
Phase2TrackerClusterizer / RecHits · host · legacy

↓

Phase2OTRecHitsSoAConverter
selects P-hits (PSP, TOB only) · host fill · H2D

↓

pixel-style TrackingRecHitSoA
same columns as pixels · moduleStart offset by nPixelHits · detectorIndex = nPixelModules + i

↓

pixel + OT P-hit SoA merged
beam spot already subtracted · iphi, rGlobal filled

↓

CA (Phase-2 OT extension) + fit
P-hits are just more pixel layers · device

Only the inner sensor of PS barrel modules is used; 2S modules and the endcap do not enter. No direction information: the CA sees a point.



VectorHit-style stubs (see later)

all OT stacks, lower × upper pairs with a direction

hltSiPhase2Clusters → hltSiPhase2RecHits
Phase2TrackerClusterizer / RecHits · host · legacy

↓

PixelSeedingOTRecHitsSoAConverter
all OT hits (PS + 2S, lower + upper) → OTRecHitsSoA · host fill · H2D

↓

Phase2OTVectorHitsSoAProducer
count / scan / form · StackedModuleGeometry condition · device

↓

CAHitNtupletAlpaka (Phase2OTVectorHits) + BrokenLine
VectorHit-aware doublets via κ from dPhiDr · device

Full OT acceptance (barrel + endcap, PS + 2S) and a direction per hit; the pair-forming cost moves onto the device.

"**I**terative **T**racking on **A**ccelerators with **CA**"



What RAW changes, per setup

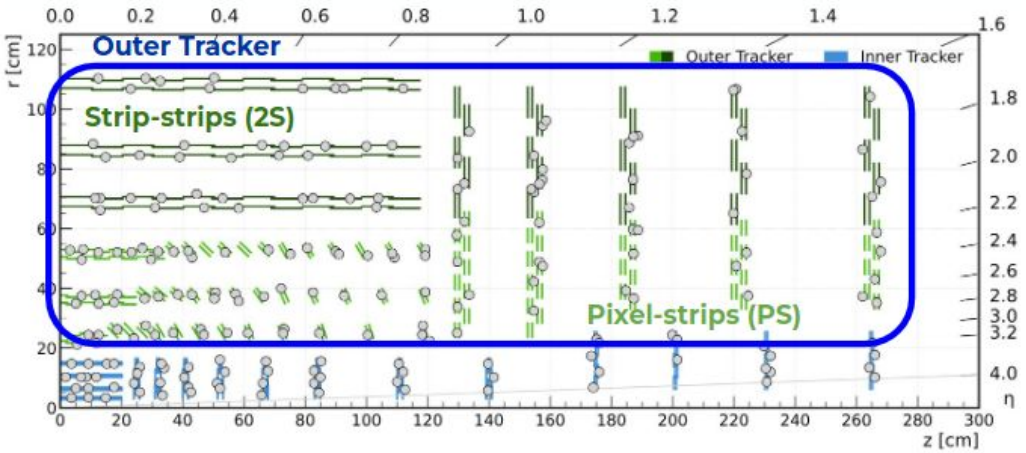
`RawToCluster`: needed by both
Every chain above starts from the legacy clusterizer on simulated digis. A device OT RawToCluster (same mould as the pixel one) replaces the first host box in both diagrams and is a precondition for any realistic timing.
Request: a timeline and a format owner.

`RawToStubs`: useful for the VectorHit-style chain
A device Raw2Stubs unpacker gives the CA its outer hits with no pairing, and the VectorHit-style former becomes the full-resolution refinement on top. Of no use to the P-hit extension, which wants single hits.

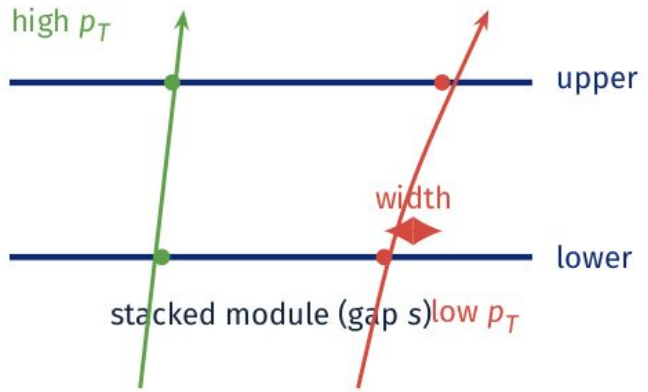
Questions to the DPG: is **`RawToCluster`** planned, on what timescale? Is **`RawToStubs`** planned, on what timescale?

A few words about Vector Hits

- Phase-2 OT modules are stacked pairs of closely spaced sensors (~mm)
  - PS → macro pixel + silicon strip
  - 2S → 2 silicon strips
- Stubs build the lower x upper cluster pair in each module
- Generic implementation based on the VectorHits already available in CMSSW (link)
- Physics reference from CMSSW (as implemented by Erica Brondolin and described in her thesis)

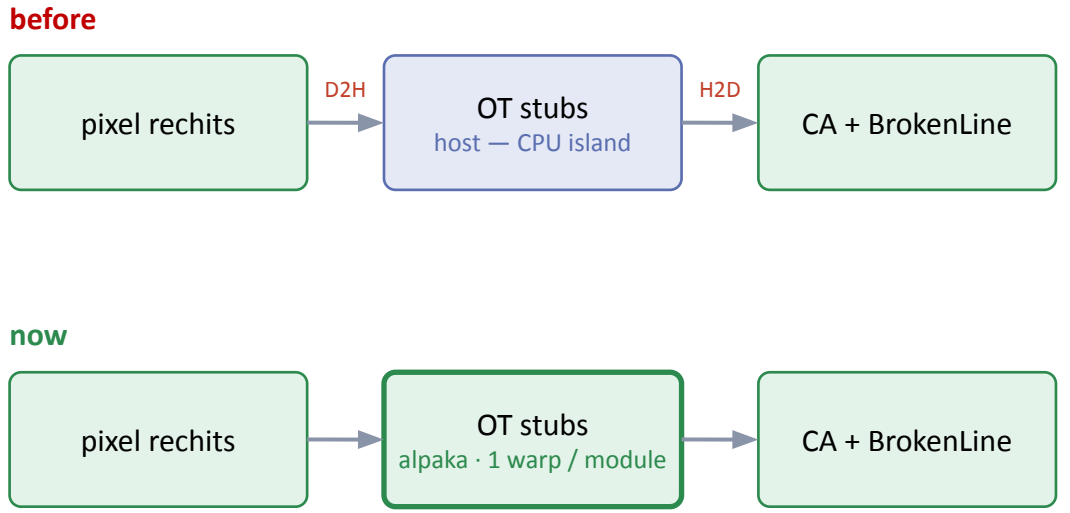


- CPU VectorHit
  - 4 parameters (dx/dz, dy/dz, x, y) from the stacked clusters
  - Two independent z–x / z–y least-squares line fits + full 4 × 4 covariance
  - Parallax correction transports the lower-sensor direction across the gap
  - Layer-dependent width cut for acceptance



Why build OT Vector Hits on GPU ?

- Phase-2 HLT pixel tracking is extended to use OT stubs as outer hits: a much longer lever arm ⇒ sharper pT , $d0$, fewer fakes.
- The pixel CA + BrokenLine fit already run end-to-end on device (alpaka). OT stubs would have been the only CPU island in that chain.
- This work moves OT stub formation onto the GPU, taking the offline VectorHit builder as the physics reference.



Only legacy OT rechits enter from the host (one SoA conversion per event); a single uint32 — the stub count — comes back to size the output.

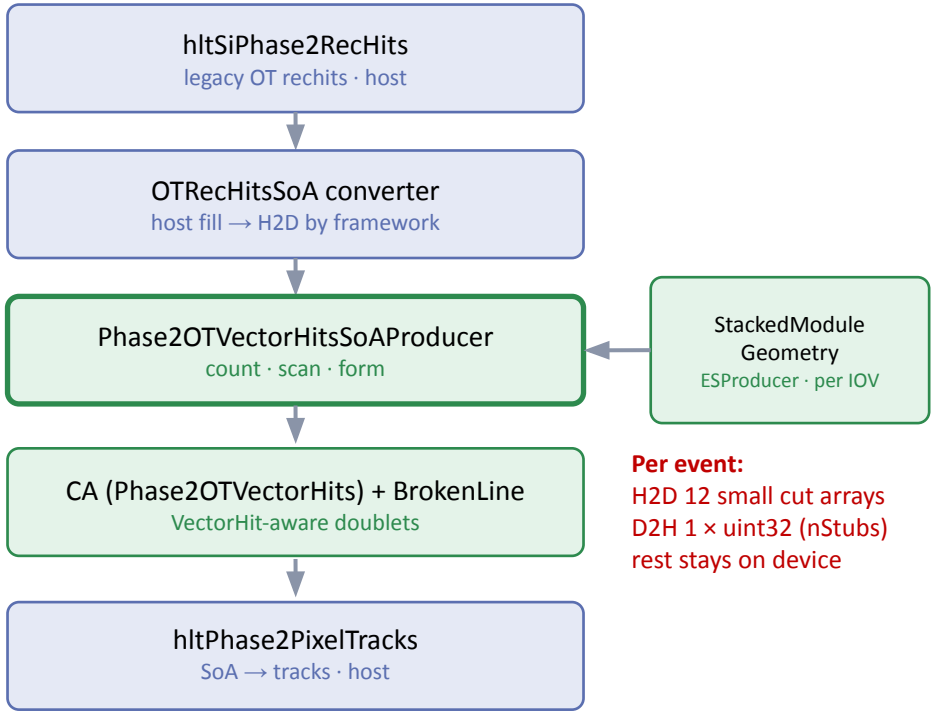
What was built (PR #52171)

- A portable count / scan / form VectorHit former (`Phase2OTVectorHitsSoAProducer`).
- Two new SoA data formats: `VectorHitsSoA` (the former's output) and `OTRecHitsSoA`, the legacy OT rechits repacked into SoA by the `PixelSeedingOTRecHitsSoAConverter`.
- A full truth-matched DQM validation suite.

How it is made heterogeneous-compliant?

Portable kernels, SoA everywhere, geometry as a device condition, two passes and one tiny copy.

- **Alpaka throughout: `Phase2OTVectorHitsSoAProducer@alpaka` runs on CUDA when a GPU is present and serially otherwise, from the same source.**
- **Structure-of-Arrays data formats. `OTRecHitsSoA` repacks the legacy rechits (positions, errors, cluster size, truth index); `VectorHitsSoA` holds only iphi, dPhiDr (+ error), two hit references and flags. Same layout host and device, moved by one memcpy.**
- **Geometry is a condition, not hardcoded: an @alpaka ESProducer bakes TrackerGeometry + Topology into `StackedModuleGeometrySoA`, one row per stacked module (separation, tilt, unit vectors, sensor frames). Row order = CA module index.**
- **Count / scan / form: pass 1 counts accepted pairs per module, a prefix scan gives offsets, pass 2 writes each module's stubs into its own slice. Output allocated to exactly nStubs; no atomics in the hot loop.**
- **One warp per module: 32 lanes stride the lower×upper pair loop; warp-reduce for the count, warp prefix-scan for deterministic output slots.**



Per event:
H2D 12 small cut arrays
D2H 1 × uint32 (nStubs)
rest stays on device

What is taken from E. Brondolin's thesis

CERN-THESIS-2017-328, §6.2 "Vector Hits": the local-reconstruction part is reused, the global part is not

Thesis element	Where	In the GPU `VectorHits`
The vector hit: **lower×upper cluster pair** on one stacked module, position from the lower cluster	§6.2.1, Fig. 6.13	**kept** — a stub is exactly this pair
Width **w = |x_lower − x_upper|** as the best pT discriminant among κ, φ, |Q|/pT, w	Eq. 6.10, Fig. 6.17	**kept** — the acceptance cut
Parallax correction **p = Δw·c_x** and its four sign cases (which cluster is shifted, which way)	Eqs. 6.11–6.17, Figs. 6.19–6.20	**kept** — ported line for line via the CPU VectorHitBuilder
Per-layer width thresholds instead of a fixed 10σ	Eq. 6.18, Table 6.4	**kept**, retuned (flat barrel 0.05–0.2 cm, tilted 0.15, endcap 0.1)
Exact 3-point circle through beam spot and both clusters: ρ, pT = 0.003 B ρ	Eqs. 6.2–6.9	**kept** as the ptEst diagnostic (not stored in the stub)
4-parameter (x, y, dx/dz, dy/dz) linear fits + full covariance	Eq. 6.1	**dropped** — replaced by one dPhiDr + error
Extended Kalman filter and a vector-hit-only seeding iteration in the CTF	§6.2.2–6.2.3	**not used** — the consumer is the CA, not the CTF

- The thesis asked whether vector hits help the offline Combinatorial Track Finder.
- The GPU work takes its answer to the local question how to tell a genuine pair from a combinatoric one on a single module and feeds it to a different consumer, the HLT pixel CA.
- Default width cuts start from Table 6.4 of the thesis (barrel 0.05, 0.06, 0.08, 0.09, 0.1, 0.12 cm; endcap 0.1 cm) and were loosened for the outer flat barrel and the tilted modules.

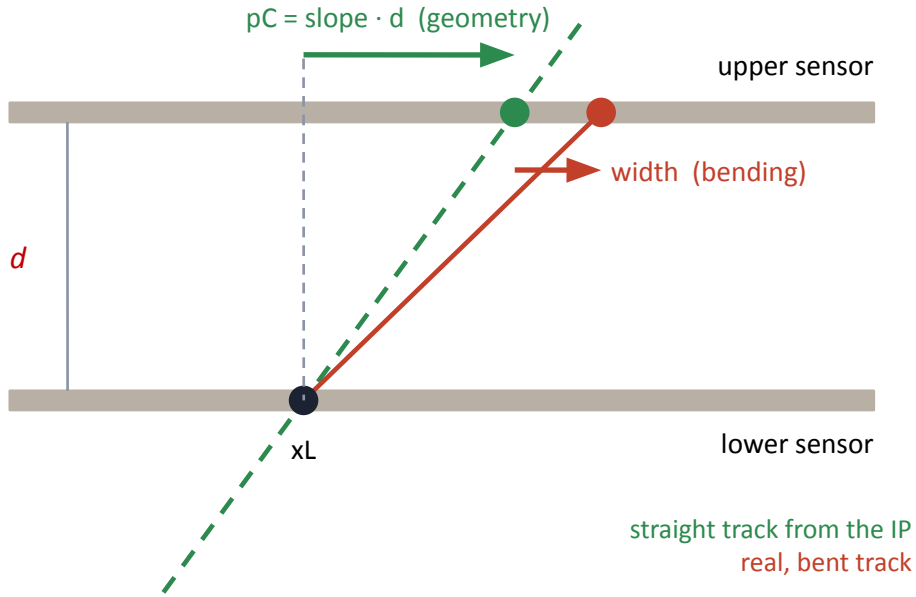
The Parallax correction

1 The problem

- A track from the interaction point hits a stacked module at an angle. The upper cluster is therefore displaced from the lower one by slope × gap even for an infinitely stiff track.
- If we used the raw cluster offset as the p_T estimator, every module away from normal incidence would look like a low-p_T track.

2 The fix: project the lower hit across the gap

- Take the direction from the origin to the lower hit, express it in the module frame, and move it across the gap d.
- The expected offset is **pC = (vx / vz) · d**.
  The p_T-sensitive quantity is then **W = (xU − xL) − pC**, accepted if **|W| < cut(layer)**.
- Same formula as the offline VectorHit builder, ported line by line.



up to 570 µm

largest pC, flat-barrel sensor edge — same size as the L1 width cut, so it cannot be skipped

0.05 µm

worst GPU − CPU difference on D110, same origin: float rounding

Median pC: 53 µm flat barrel, 94 µm tilted barrel, 55 µm endcap PS, 25 µm endcap 2S.

How dφ/dr is used in the CA tracking

One number per stub, dPhiDr, is turned into a track curvature that the CA compares between hits before linking them

1. **A stub carries one direction number: dPhiDr = Δφ / dr_eff**
Δφ is the azimuthal offset between the two clusters after the parallax correction; dr_eff is the radial span the track crosses between the sensors, **s / (cos θt + sin θt · z/r),** so tilt and position along z are already folded in. Its error comes from the two hits' φ errors added in quadrature.

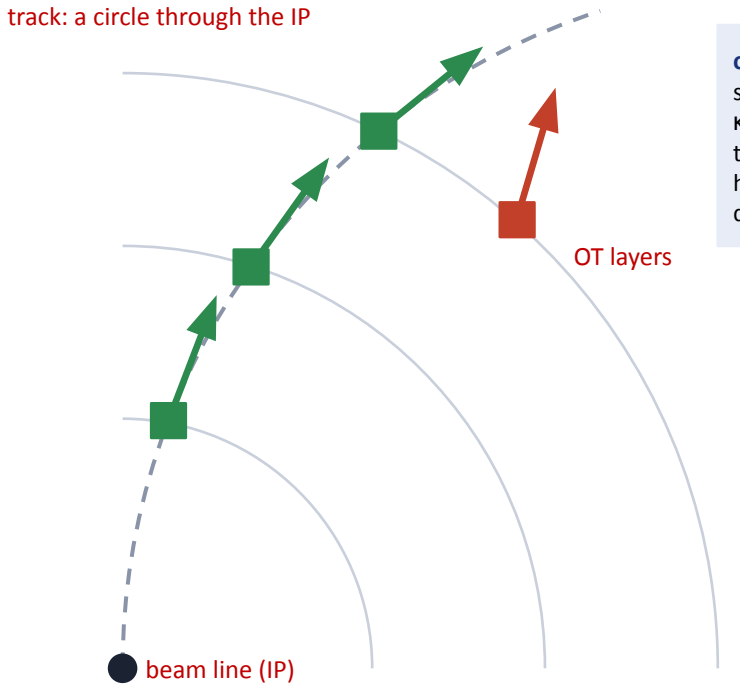
2. **A direction at radius r fixes the circle through the IP**
For a circle through the origin the angle ψ between the track and the radial direction obeys **sin ψ = r/(2R)** (tangent–chord theorem), and **tan ψ = r · dPhiDr**. Eliminating ψ gives the half-curvature **κ = 1/(2R)** and its error σκ from σ(dPhiDr). κ is the same at any r on one track, so stubs from different layers are directly comparable.

3. **The CA cuts on the κ difference when it builds doublets**
Stub–stub**: $|\kappa_i - \kappa_o| / \sqrt{(\sigma\kappa,i^2 + \sigma\kappa,o^2)} >$ stubSigmaCut** → the doublet is rejected.
Pixel–stub: the curvature of the pixel–stub pair itself is compared to the stub's own κ, and the sign of dPhiDr must match the doublet's bending direction (sameDPhiSign). The cut value is set per layer pair (maxStubCurvSigma).

4. **What it buys**
Each stub acts as a short track segment: genuine stubs on one track share κ within errors, mis-paired stubs carry a random dPhiDr and are dropped at the first link. Fewer doublets survive into the ntuplet building, the fake rate drops, and the κ comparison replaces the bend / pT estimate that the stub no longer stores.

genuine stubs: every arrow is tangent to the same circle, so every stub gives the same curvature → the CA links them

mis-paired stub: right place, wrong arrow, so its curvature disagrees with its neighbours → dropped at the first link



one knob
stubSigmaCut: the κ-significance the CA tolerates between two hits; per layer pair, < 0 disables

Transverse view, bending exaggerated.

Validation in brief

Truth-matched against `TrackingParticles` through the Phase-2 digi SimLinks not against `TTStubs` or offline VectorHits.

- **Classification of each stub by the TP sets of its two hits: genuine (common particle), combinatoric (no common TP), unknown (no sim links), PHitOnly (position-only PS recovery stub).**
- **Position residuals vs the matched TP (straight-line extrapolation) are centred at zero with σz ≈ 0.6 cm, in clean single-μ and under 200 PU alike.**
- **`dPhiDr` significance separates genuine from fake at 10–100 GeV; at 1 GeV and under PU the CA's two-stub κ comparison does that work instead.**

~92 %

stub efficiency, single μ at 1/10/100 GeV — flat in η and pT

≲ 1 %

fake rate, single μ

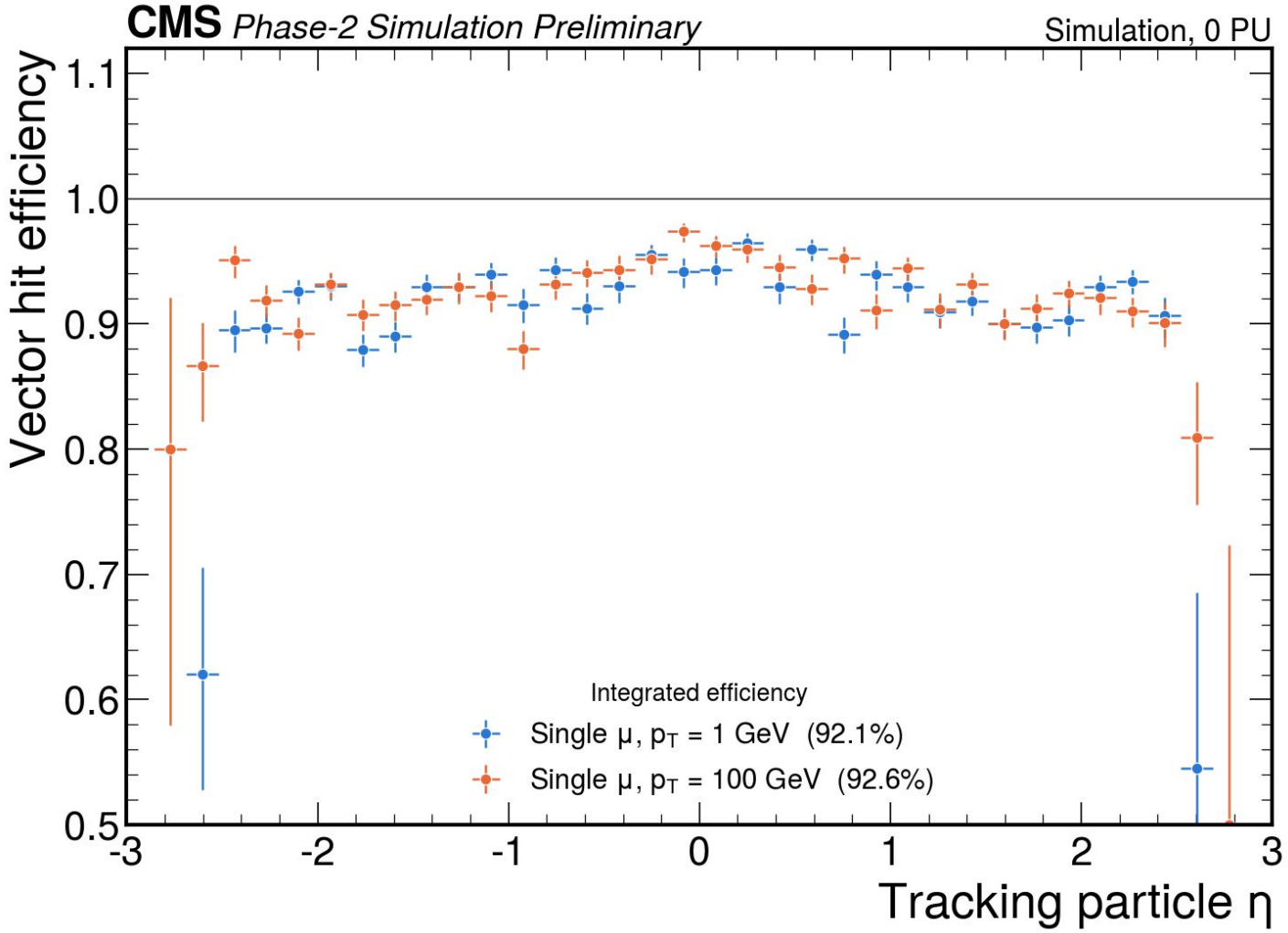
~80 %

efficiency in $t\bar{t}$ + 200 PU (barrel 81 %, endcap 84 %), ⟨N_stubs⟩ ≈ 68k / event

58 / 42

genuine / combinatoric per raw stub; mis-pairings peak in the outer 2S barrel, removed by the CA κ-cut

Selected Validation results (single μ p_T=1,100 GeV)



92.1%

pT = 1 GeV

92.6%

pT = 100 GeV

integrated over all layers and disks

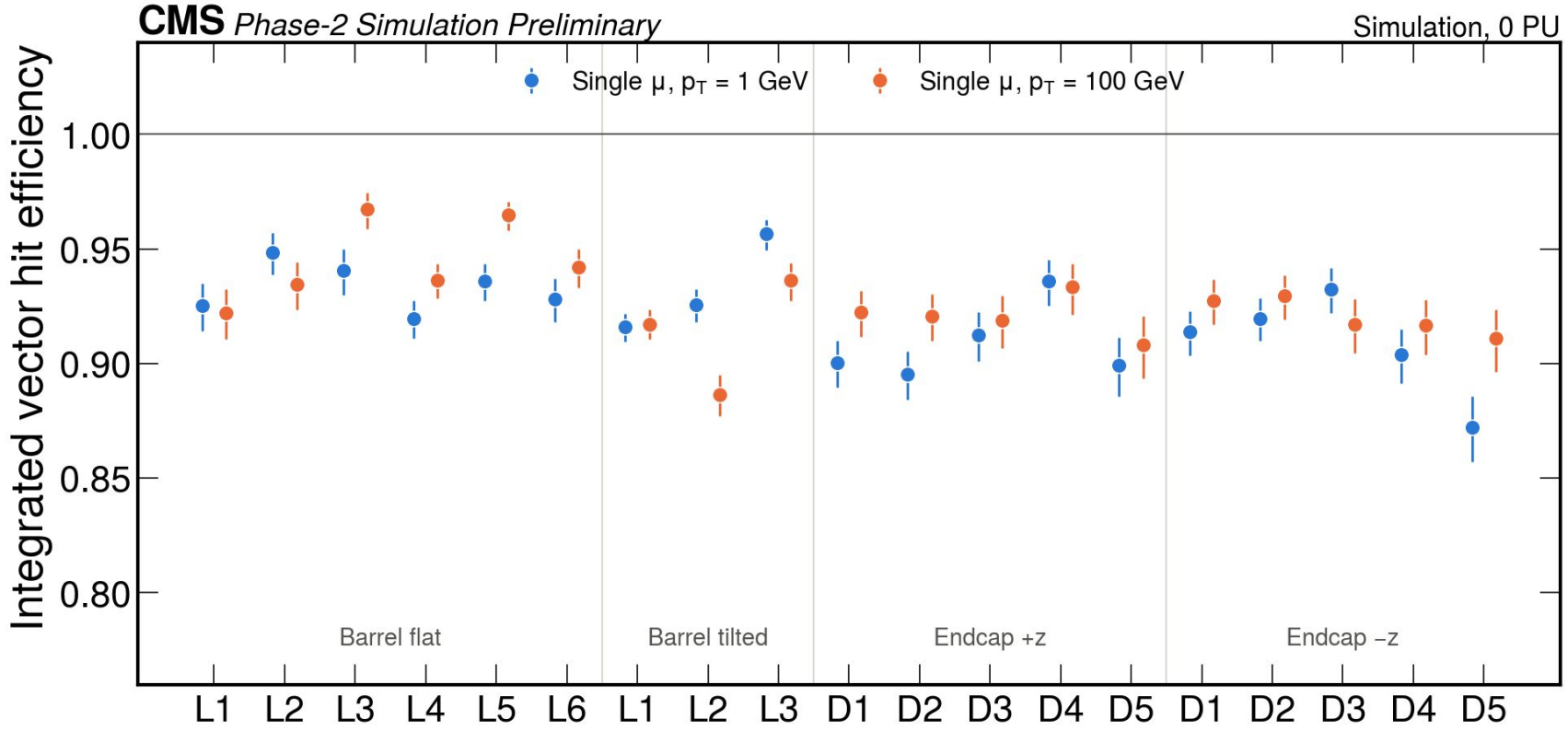
1. **No pT dependence**

   The two samples agree within errors at all η

2. **Flat up to |η| ≈ 2.4**

   Drops only at the edge of the OT acceptance

Selected Validation results (single μ p_T=1,100 GeV)

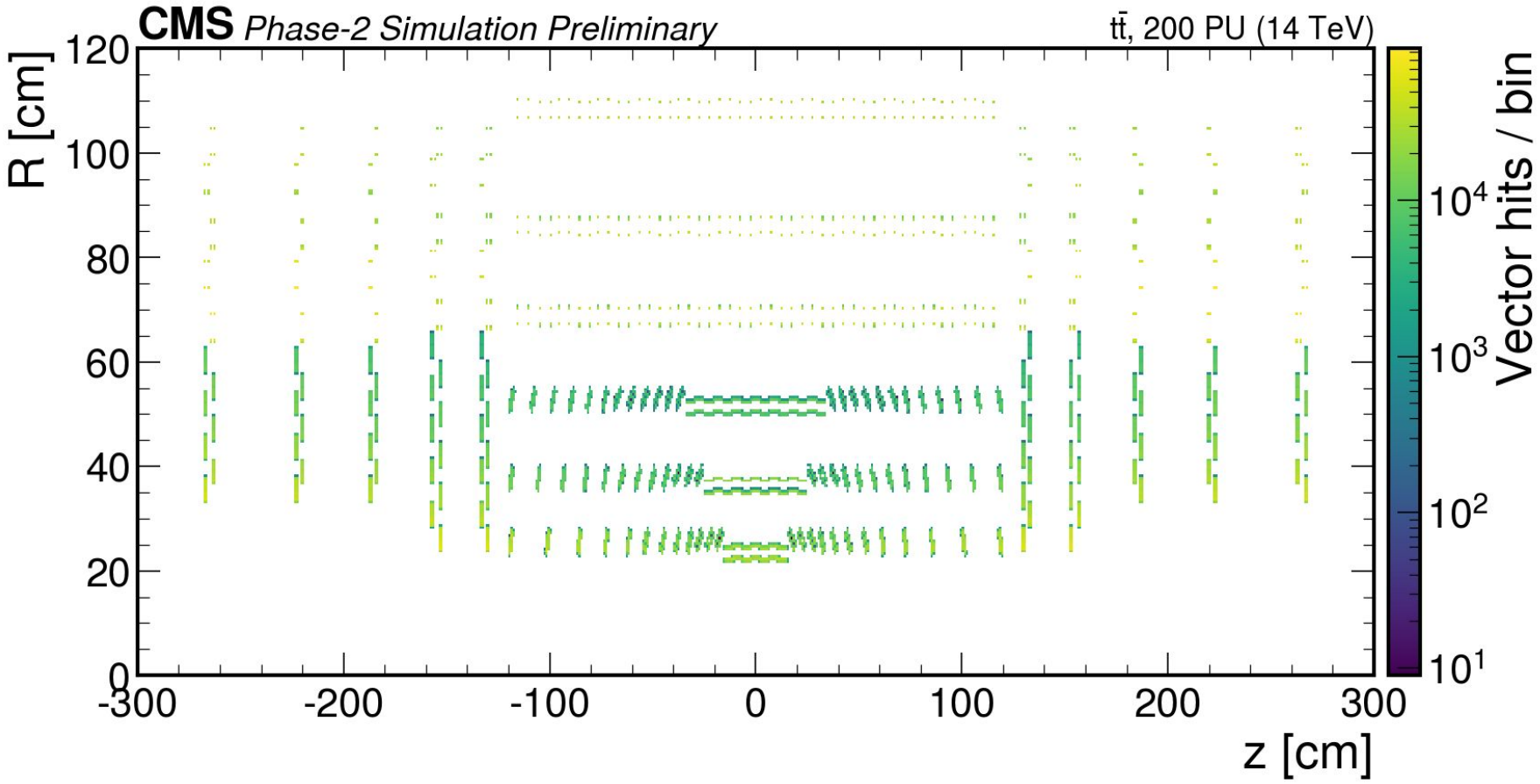


1. **Barrel flat**
   92-95% (1 GeV)
   92-97% (100 GeV)

2. **Barrel tilted**
   89-96%, lowest in L2 at 100 GeV

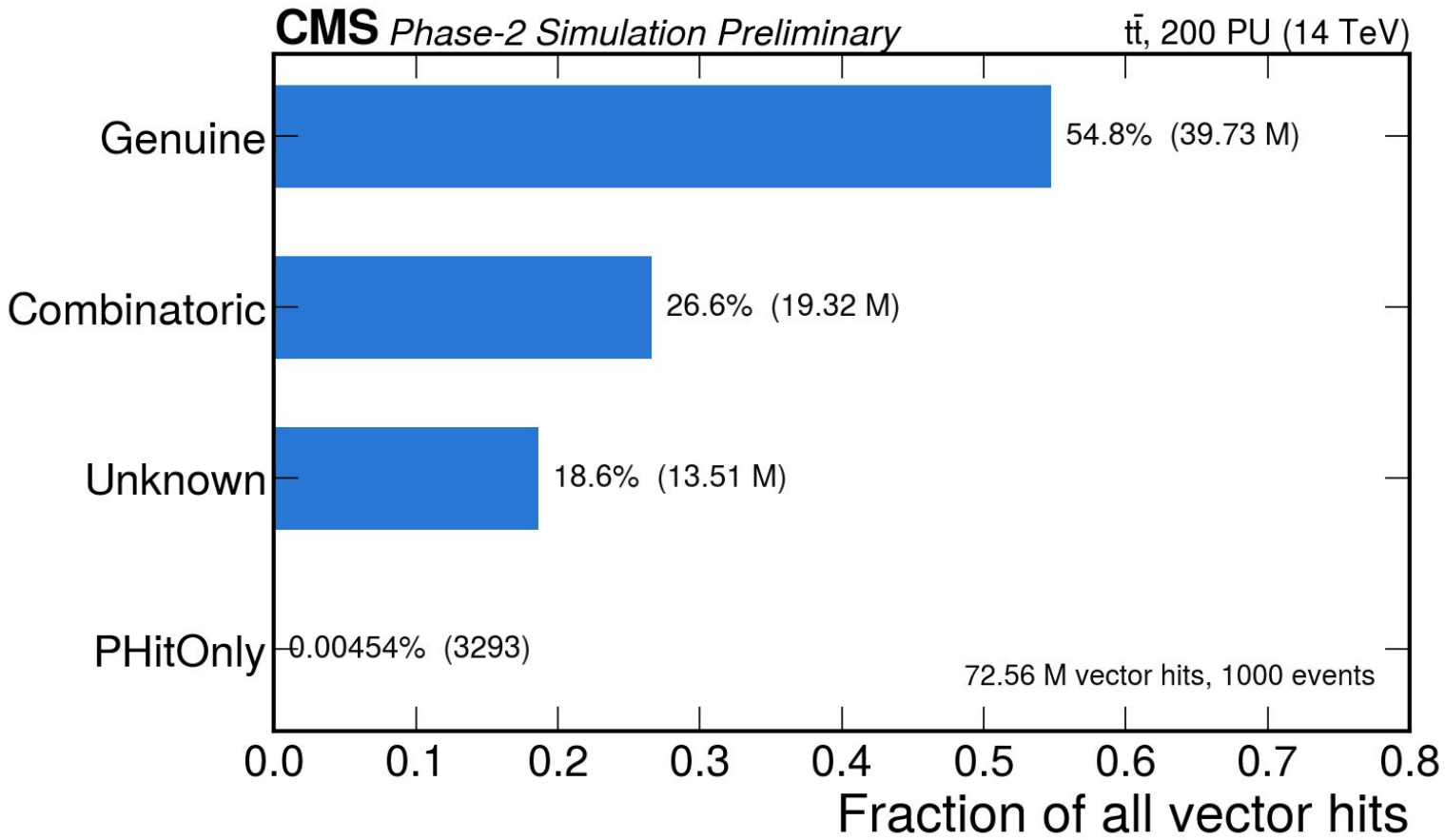
3. **Disks**
   87-94%, lowest in D5 -z at 1 GeV

Vector-hit r–z map in $t\bar{t}$ 200 PU



Barrel: 6 layers, R ≈ 22-112 cm, tilted rings in L1-L3. **Endcaps:** 5 disks per side, |z| ≈ 130-270 cm.

Selected Validation results ($t\bar{t}$ + 200 PU)



72.6k

vector hits per event (RMS 8.7k)

45%

not genuine: 26.6% combinatoric, 18.6% unknown

98%

genuine in single-muon samples, for comparison

Conclusions & Outlook

Where we stand

- **The Run-4 HLT menu lives in cmssw** (HLT_75e33), reviewed by STORM through PRs, with no confDB counterpart yet. It is a few tens of paths built to exercise object reconstruction, not a physics menu; about half of the L1T seeds are unused.
- **The IT local reco is portable** (digi → cluster → rechit on device, Patatrack-style); the OT one is still the legacy clusterizer and rechit producer on the host, converted to SoA for the CA.
- **OT Vector Hits now form on the GPU** (PR #52171): a count/scan/form former on OTRecHitsSoA, geometry as a device condition, one uint32 D2H per event. Physics taken from the offline VectorHit builder; parallax agrees to 0.05 μm on D110.
- **Validated against TrackingParticles:** ~93 % stub efficiency and ≲1 % fakes for single μ, ~80 % in ttbar + 200 PU, unbiased residuals and calibrated errors.
- **Everything still runs from GEN-SIM-DIGI,** not RAW: no unpacking cost, no realistic data volume in any timing number shown today.

What we ask of the Tracker DPG

- **RAW format and packer/unpacker** for IT and OT, with an owner and a timeline against the Computing CDR (first version due EOY 2026) and CSA28. A device `RawToCluster` is needed by both OT chains; `RawToStubs` would add value for the Vector-Hit chain
- **Shared ownership of the portable local reco:** the DPG owns the physics (CPE, thresholds, Lorentz angle), NGT/HLT the implementation. We need regular HLT-vs-offline comparisons that runs, so the next PR #52174 is caught by validation rather than by chasing performance.
- **A portable OT clusterizer + CPE** writing `OTRecHitsSoA` directly, removing the legacy collections and the host converters from the HLT path.

Next steps on our side: integrate the Vector-Hit CA into the timing menu and quantify the gain in pT, d0 and fake rate against the P-hit extension; track the open technical issues (FEVTDEBUGHLT definition, EventSetup labels, unseeded paths) consistently.


CERN
NexTGen
Next Generation Triggers

STORM and its role within TSG

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **It is a subgroup of TSG, whose mandate is integrating code for physics object reconstruction from POGs / PAGs / DPGs in order to implement HLT algorithms.**
  - Integrate new triggers in the HLT, and provide menus for all configurations (p-p collisions, cosmics, Monte-Carlo, heavy-ions, special menus for tests). The deployment of those menus online is then the task of the FOG.
- **The group ensures that:**
  - each HLT algorithm satisfies the modular structure of the framework (L1 preconditions, data-unpacking, producers, filters, book-keeping) and that it can run in the trigger menu with or without other HLT paths.
  - the information stored by HLT for tagging and book-keeping purposes will comply with the (online, FEVT, *AOD) size and performance requirements and will gradually evolve according to feedback received by the Offline, Physics and Trigger groups.
  - the resulting data streams are usable by the POGs ands PAGs.



Martin Grünewald (UCD)



Marco Musich (CERN)

CMSSW and its release cycles

- **CMSSW (CMS SoftWare): cms-sw.github.io**
  - Main software package used by CMS for simulation, calibration, reconstruction [0]

- **CMSSW development is organised in release cycles [1] [2],**
  - e.g. **`CMSSW_20_1_X`** , **`CMSSW_20_0_X`** , **`CMSSW_17_0_X`** , etc

- **The latest cycle is the "development cycle" (master branch)**
  - development lasts O(months), with a deadline usually set based on specific purpose, e.g. **`20_0_X`** cycle for the 2026 Phase-2 MC production;
  - pre-releases (e.g. **`20_1_0_pre1`**, **`_pre2`**, etc) are cut every few weeks leading up to the first full release of a cycle, e.g. **`CMSSW_20_1_0`**;

- **Once a development cycle closes, the master branch of CMSSW moves to the next cycle;**
  - recently-closed cycles can still receive updates (backports), but those have to be justified (necessary new features, bug-fixes), and new releases are made on demand (e.g. **`20_0_0`** → **`20_0_0_patch1`**)

Proposing changes to CMSSW

- **Changes to CMSSW are done via "pull requests" (PRs) to the CMSSW central repository on GitHub**
  - First PR to master branch, then backports to earlier cycles if needed (and allowed)
  - info on how to make a PR to CMSSW (incl. coding rules)
  - if unsure, ask the relevant POG/PAG contacts or TSG/STORM
  - examples: [add new plugin] [update existing plugin]

- **TSG / STORM maintains a curated digest (pre-) release by (pre-) release at this Twiki Page**

About CMSSW packages

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **Source code of CMSSW is organised in "packages": `[SubSystem]/[Package]`**
  - Reviewers of a PR are chosen based on which packages are modified by the PR.

- **Most of the HLT packages of CMSSW are under HLTrigger/*/**
  - countless examples of typical filters used at HLT (the plugin you need might exist already, check here first).

- **Most of the reconstruction-related producers used at HLT come from the offline reconstruction, and they are not maintained by TSG:**
  - If your PR does not touch HLT packages, but is relevant to HLT, please notify TSG, and tag **`@cms-sw/hlt-l2`** (Martin and M.M.) in your PR.

Running the menus and testing

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **General instructions to be able to run one of the Phase 2 menus are collected here.**
  - Typical recipe requires to re-run the (lightweight) L1P2GT step in cmsDriver and to specify **conditions, geometry and cmssw era**.

```
cmsDriver.py Phase2 -s L1P2GT,HLT:<menu name> --processName=HLTX \
--conditions auto:phase2_realistic_T35 \
--geometry ExtendedRun4D121 \
--era  Phase2C22I13M9\
--eventcontent FEVTDEBUGHLT \
--customise SLHCUpgradeSimulations/Configuration/aging.customise_aging_1000 \
--filein file:output_Phase2_L1T.root \
--inputCommands='keep *, drop *_hlt*_*_HLT, drop
triggerTriggerFilterObjectWithRefs_l1t*_*_HLT' \
--mc \
-n 1 --nThreads 1
```

- **Few tools have been introduced to be able do basic integration tests and basic computing profiling:**
  - **hltPhase2UpgradeIntegrationTests** : testing script to make sure that each path is self-contained and reproducible across trials, run by default in IBs, can be used on demand in PRs by cms-bot (via "`enable = hlt_p2_integration`");
  - **runHLTTiming** : script to prepare and run timing benchmarks, run by default in IBs, can be used on demand in PRs by cms-bot (via "`enable = hlt_p2_timing`");

HLT specific plugins and data formats

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

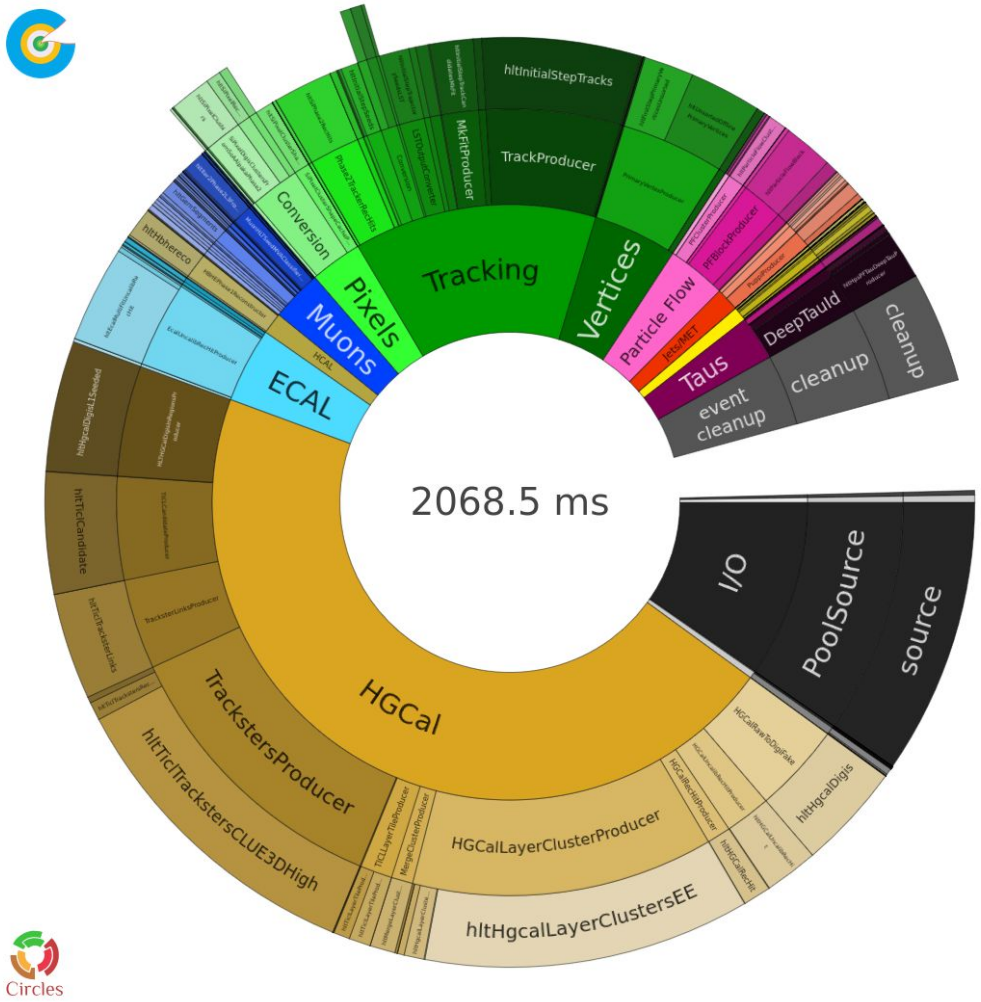
- **HLT provides a special filter type named `HLTFilter` which (inherits from `EDFilter` and) can add to the Event refs to products (physics objects) passing user-defined criteria**
  - refs created by `HLTFilters` can be used by other filters/producers
  - many filters used at HLT inherit from **`HLTFilter`**, to use this functionality.

- **These refs, and a light-weight version of these products (`TriggerObjects`, p4+id), can be written to output with HLT-specific data formats:**
  - `TriggerEvent` data format: contains **`TriggerObjects`** (saved in RAW data tier, e.g. output of HLT online)
  - `TriggerEventWithRefs` data format: contains **`TriggerObjects`** and refs to the original products (kept in larger data tiers, e.g. **`FEVTDEBUGHLT`**)

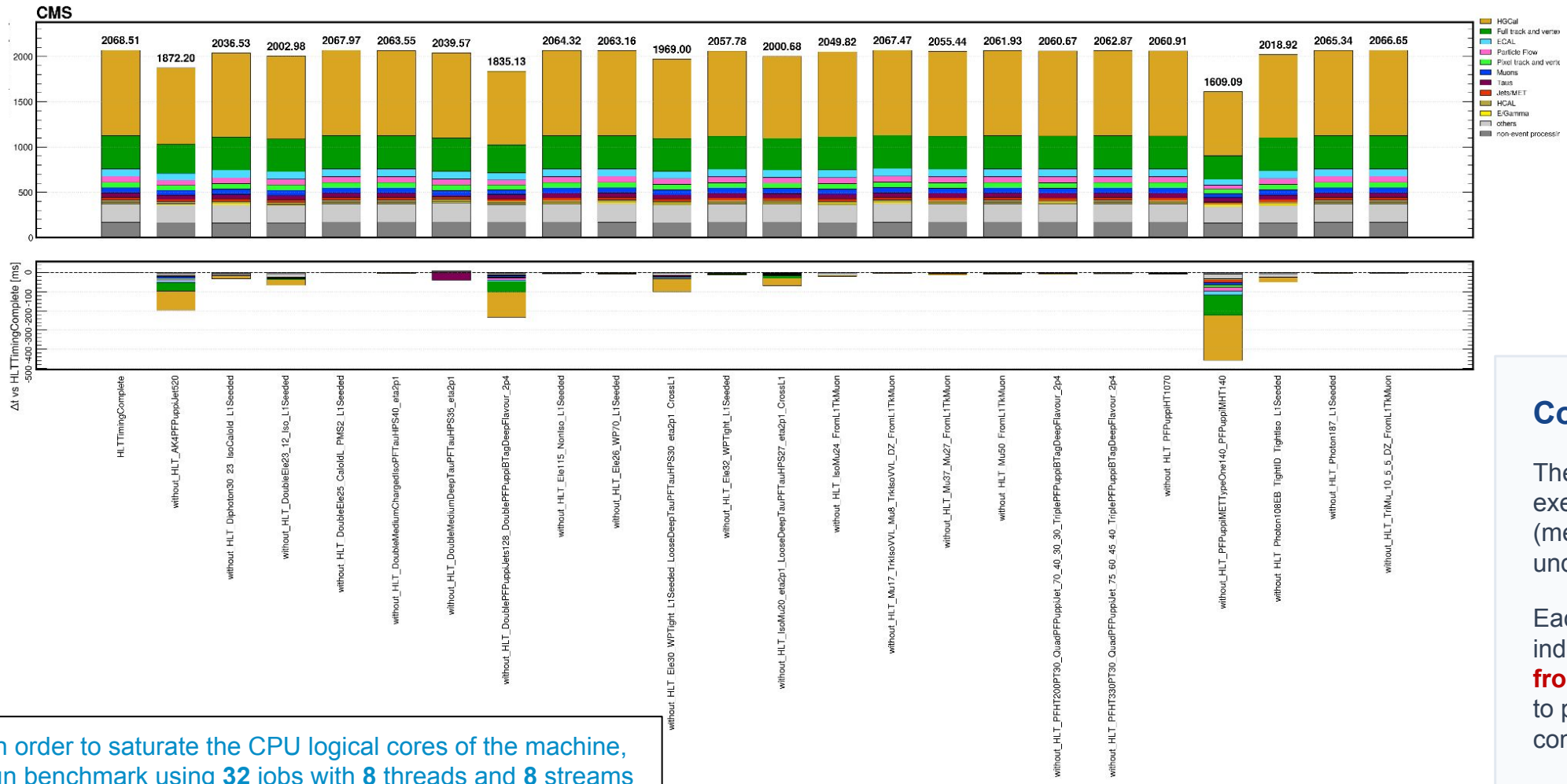
- **More details in this twiki (old twiki, but still a useful reference)**

Estimate of computing cost per path

- **Interest in understanding bottlenecks and hotspots in the current HLT menu:**
  - e.g. understand if some path is more expensive computing-wise than others or many paths are in the shadow of each other;
- **Typical timing benchmark gets executed on TTbar + PU=200 events (in automatized tests and also in private benchmarks, see more details in L. Ferragina's contribution).**
  - For this exercise, decided to use instead a skim of MinimumBias + PU=200 produced in `CMSSW_20_0_0_pre1`, retaining only the events that pass at least one of the L1T seeds currently used in the HLT menu (see previous slide).
  - Benchmark was executed on a dedicated machine equipped with 2 CPU processors AMD EPYC 9534 for a total of 256 Cores (2 sockets × 64 physical cores × 2 logical cores) and 4 NVIDIA L40S GPU



Computing cost per path



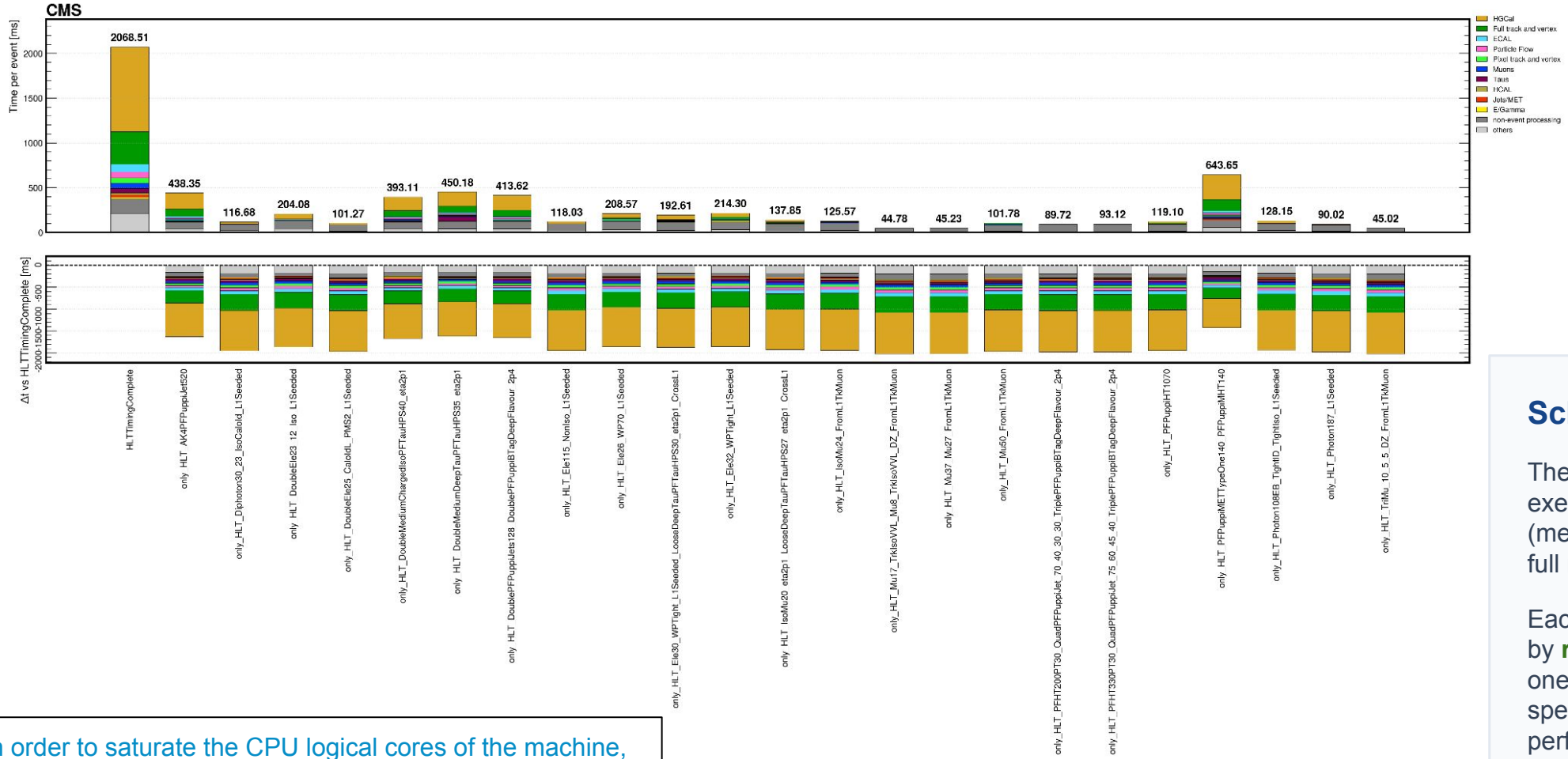
In order to saturate the CPU logical cores of the machine, run benchmark using **32** jobs with **8** threads and **8** streams

Cost Analysis

The chart illustrates the active execution time per event (measured in milliseconds) under full menu conditions.

Each path is evaluated individually by **excluding it from the schedule** one-by-one to precisely isolate its specific computing footprint.

Computing cost per path



In order to saturate the CPU logical cores of the machine, run benchmark using **32** jobs with **8** threads and **8** streams

Schedule Analysis

The chart illustrates the active execution time per event (measured in milliseconds) under full menu conditions.

Each path is evaluated individually by **retaining it in the schedule** one-by-one to precisely isolate its specific computing cost and performance footprint.

Known technical issues in the menu

STORM
SOFTWARE, TOOLS, ONLINE RELEASES AND MENUS

- **Setting aside the shortcomings in the interface with the L1P2GT discussed previously, there are some other areas which would profit of some "technical attention":**
  - The definition of the `FEVTDEBUGHLT` output data-format (used in most workflows) is currently scattered in a miriad of subsystem configuration fragments (mostly outside of the **hlt** signature area). It would be good to centralize the definitions;
  - EventSetup modules (`ESProducer`s and `ESSource`s) are either directly impoprted from offline reconstruction (technical debt) or duplicated, but without changing the producer label (i.e. potentially conflicting with offline ones, if HLT and RECO steps are run together). This is a chronic issue, since the times of Run 1.
  - A few paths use the `PathStatusFilter` to select on the L1T seed. This module is a regular `EDFilter` and not an `HLTFilter`, thus it does not leave trace of the L1T objects used in the seed in the `edm::TriggerEvent` branches.
  - Some muon-related paths run the global TICL reconstruction for isolation, thus causing a penalty throughput-wise, for no good physics reasons. Usage of dedicated local L1-seeded reconstruction sequences would help to reduce the overall timing of the menu.
  - Dependency of the menu on `GEN-SIM-DIGI` info as opposed to **`RAW`** only - in turn needs **raw2digi and digi2raw** for all subdetectors incl L1T (bits and object candidates)!

- **This is just a short list of issues that came up recently when discussing with colleagues, probably POGs are aware of others. It would be good to start tracking them consistently, as done for Run 3 menus.**
  - Some are tracked as cmssw issues in gitHub.
  - If you are aware of other technical issues, please feel free to send them our way.

Additional issues & points of concern

- **`FastSim` - who actually asked for this (for Run-3) (cf. PR cmssw#51516)?**
  - TSG dropped support for it a long time ago as the HLT menu customization routine needed is rather complex and involved.
    - Is this something needed for Phase-2? If so, for Phase-2 the groundwork should already be part of our Phase-2 HLT menu (eg, encapsulating sequences to be replaced by `FastSim` ones etc. with module labels preserving down stream `InputTags`)

- **Question to users:**
  - What tools should be improved?
    - explicit talk on new ConfDb from S. Harper, but maybe other tools cumbersome for proponents?

- **Question to trigger proponents to get their path into the menu:**
  - Is the current workflow OK (ie, trigger train model with required information etc.), or is there desire for changes/improvements?

- **Questions for Real-time datasets group about the dataset definition in terms of a) HLT paths (physics) and b) splitting of datasets and streams based on rates/bandwidth (technical issue):**
  - a) could remain as part of the HLT menu, but b) should preferentially be dealt with elsewhere. Where and how to make sure it is sound given TSG experience with it ?

Some ideas for improvement

- **As the work on the phase 2 HLT is ramping up, it would be good to start a few good habits:**
  - In general we need to strengthen the links between L1T menu and HLT menu teams;
    - Timing-wise TSG and L1T meetings are often overlapping, making direct engagement difficult.
  - It would be good to (re-)start tracking our s/w issues (and integration requests!) in a more thorough way
    - Personally I would be glad to ditch JIRA in favour of e.g. gitlab or github based issue tracking systems, but a consensus needs to be reached
  - As Run 3 becomes a part of our collective past, the boundary between the HLT Upgrade and STORM coordination areas needs to be re-defined, especially in terms of responsibilities.

Vector Hits purity in $t\bar{t}$ + 200 PU

72.6k

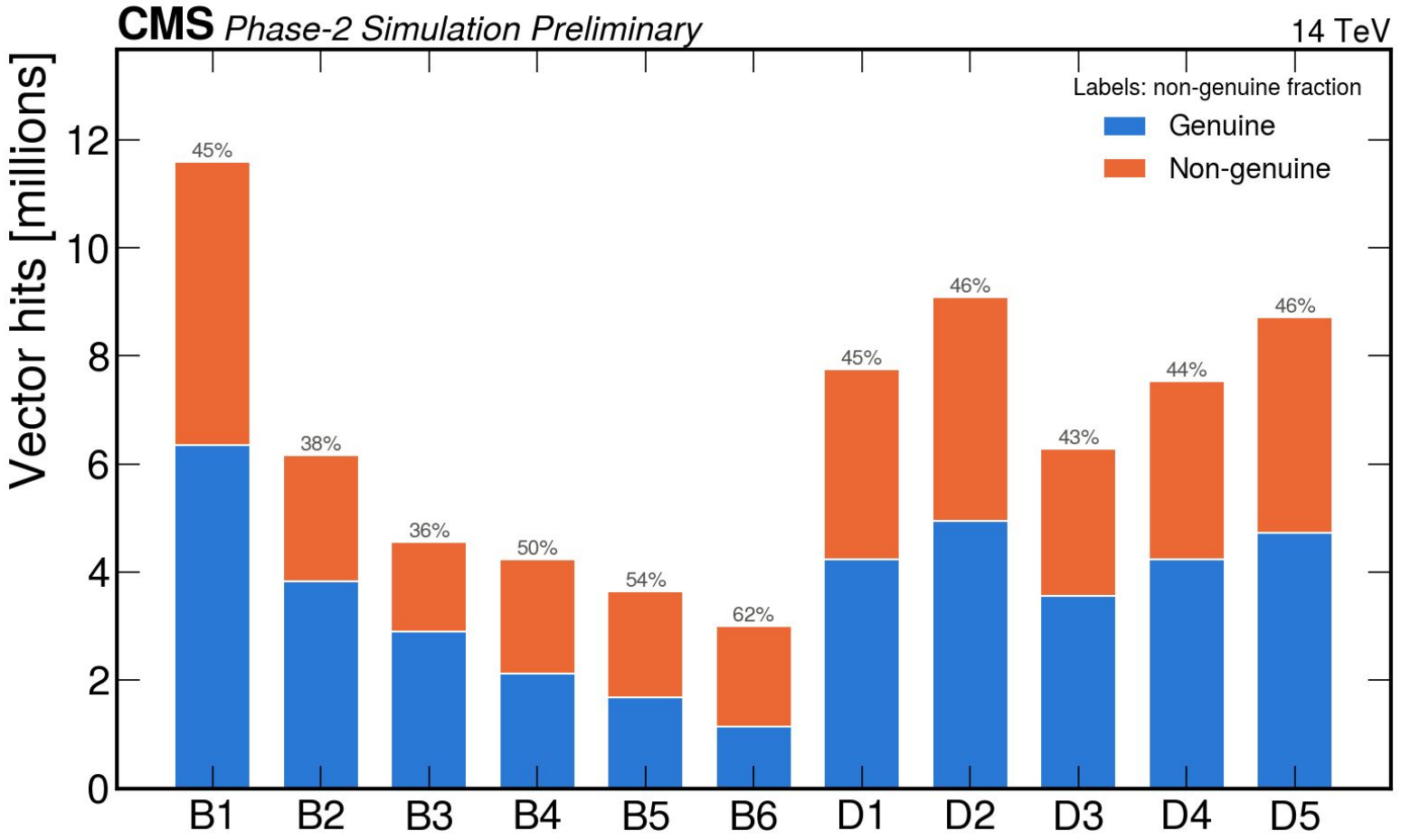
vector hits per event (RMS 8.7k)

54.8%

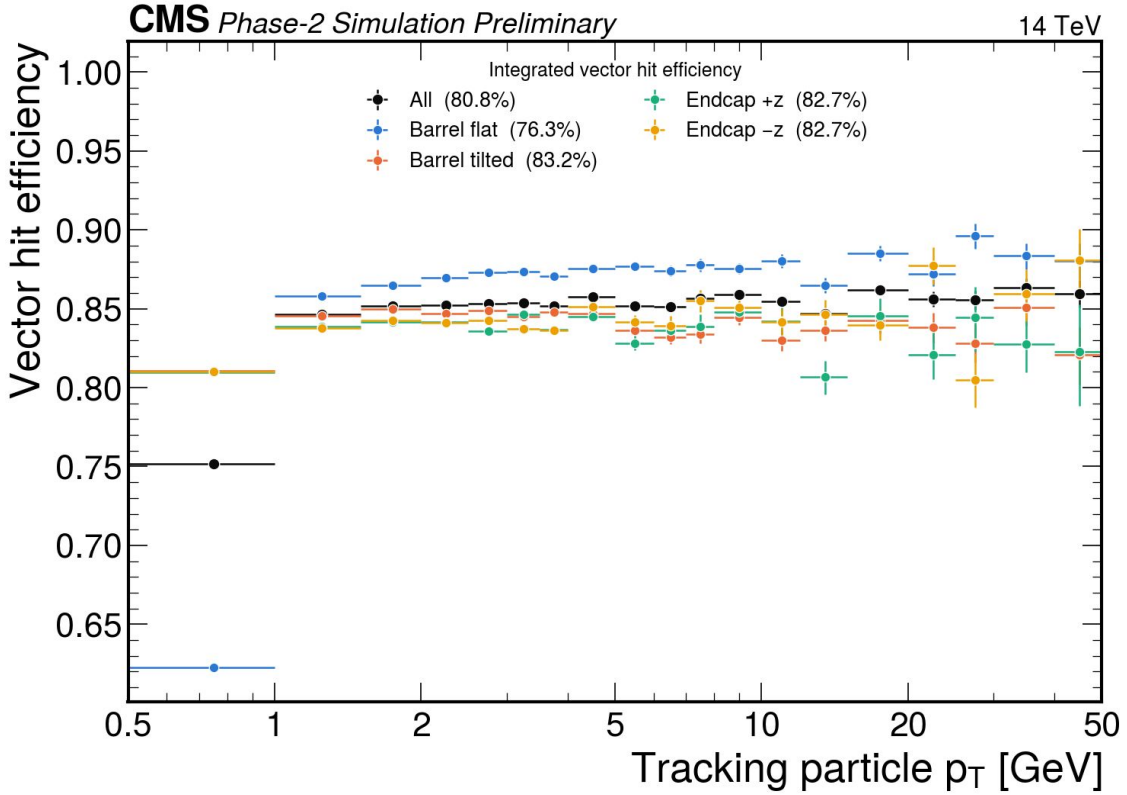
genuine · 26.6% combinatoric
18.6% unknown

80.8%

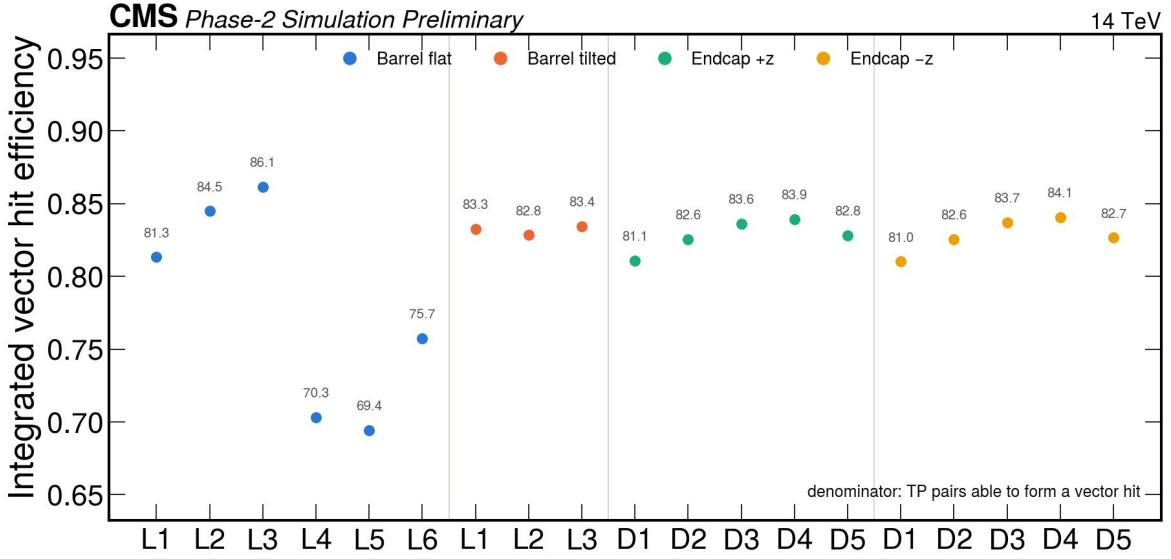
integrated vector hit efficiency



Vector Hits efficiency in $t\bar{t}$ + 200 PU



Integrated: all 80.8% · flat 76.3% · tilted 83.2% · endcaps 82.7%



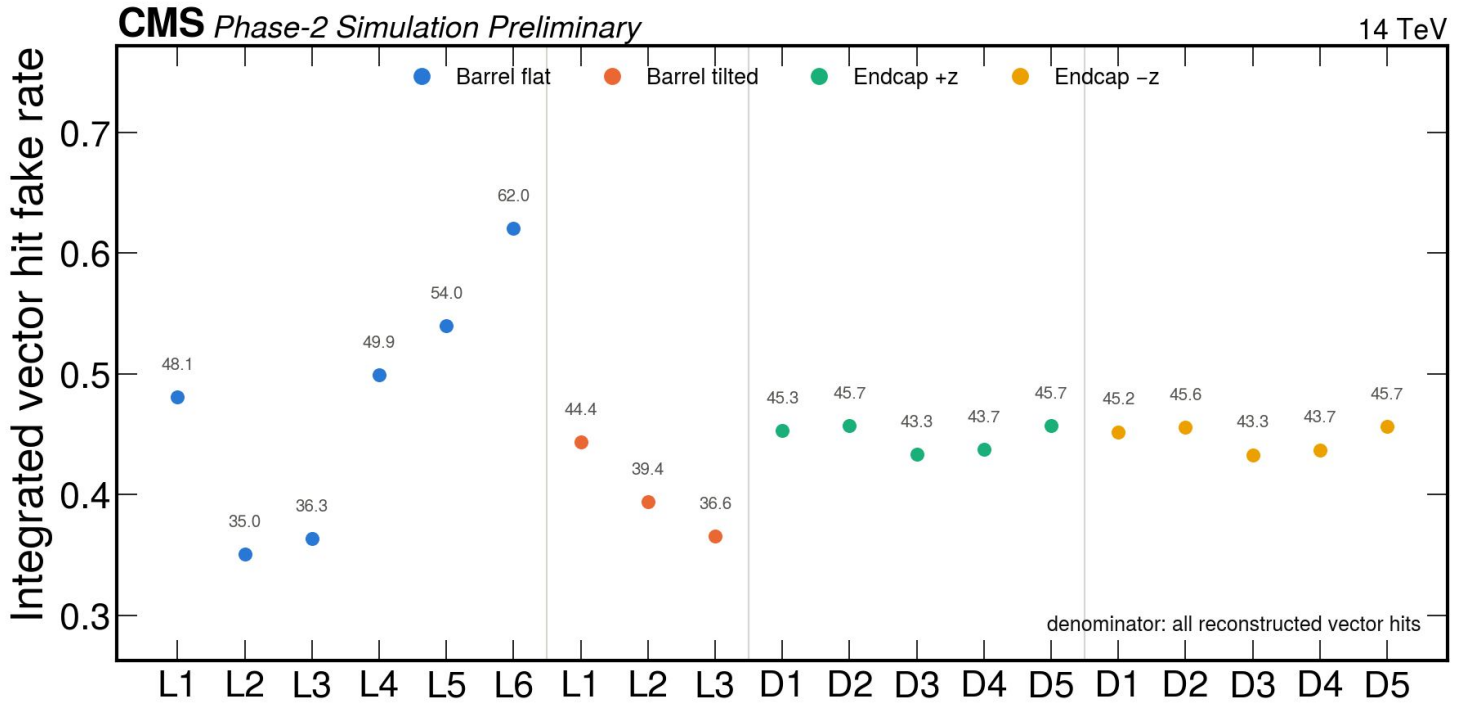
1 **Flat above 1 GeV at ~84-88%**

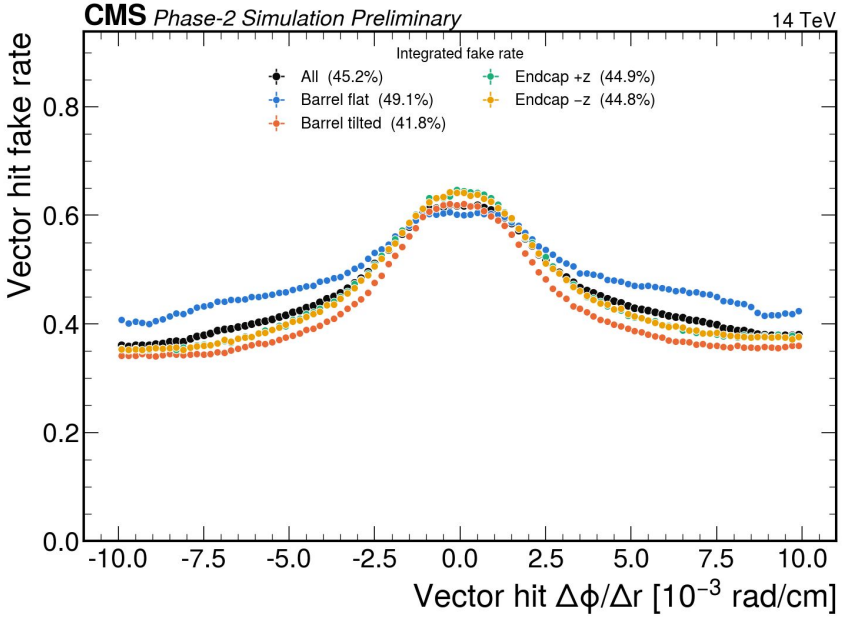
The 0.5-1 GeV bin drops to 75% (62% barrel flat) and dominates the integral

2 **Barrel flat L4-L6 (2S): 69-76%**

Tilted layers and disks are uniform at 81-84%

Vector Hits fake rates in $t\bar{t}$ + 200 PU





1 **Outer barrel is worst**

Flat L4-L6: 50%, 54%, 62%

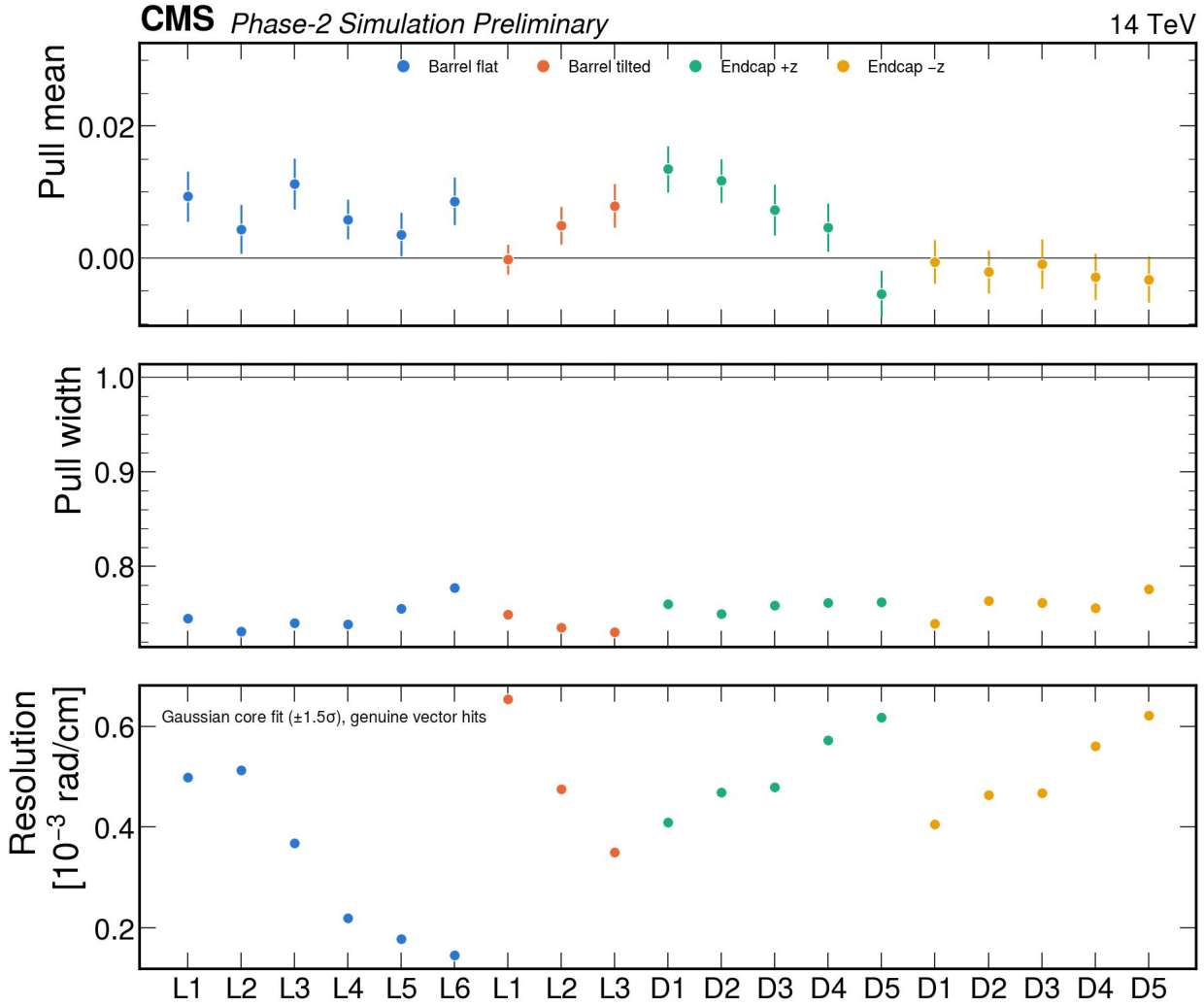
2 **2S combinatorics**

Up to 1.7 fake vector hits per lower-sensor hit in L6

3 **Peaks near zero bend**

60-65% at small $|\Delta\varphi/\Delta r|$, 35-40% at large bend

Δφ/Δr pulls



0.73-0.78

fitted pull width in all 19 layers and disks (expected 1)

1. **σ(Δφ/Δr) overestimated by ~30%**

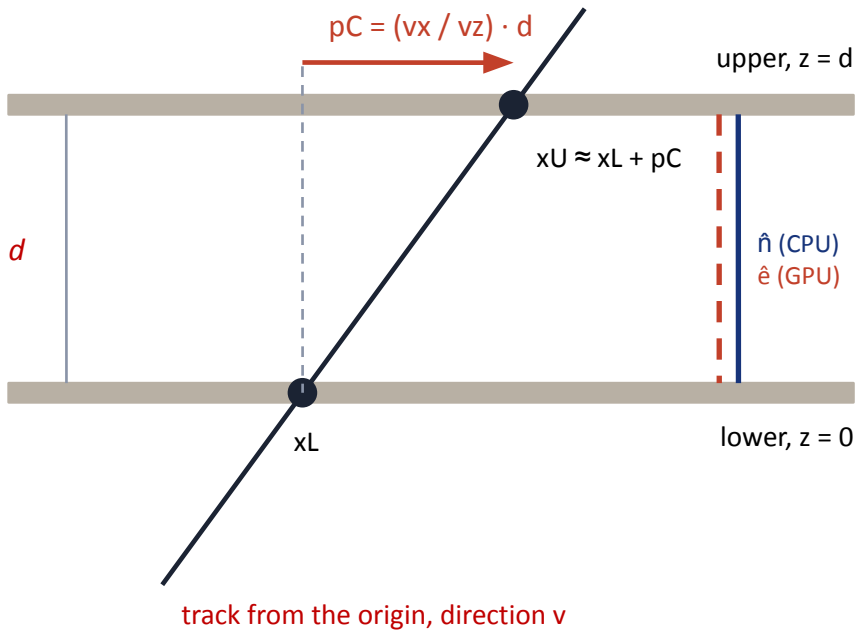
   Same in barrel and endcap, so likely a global factor in the error model

2. **Bias below 0.015σ**

   Resolution 0.15-0.65 × 10^{-3} rad/cm, best in outer flat barrel

The Parallax correction

A straight track from the origin reaches the upper sensor shifted by slope × gap; subtract it so only real bending is left



width = (xU − xL) − pC, |width| < cut_layer

	CPU VectorHit	GPU stubs
track direction v	lower hit seen from (0,0,0)	lower hit seen from the beam spot (SoA already shifted)
normal	z-row of the surface rotation	unit centre-to-centre vector × flipSign
in-plane axis	x-row of the rotation	lower local x, Gram–Schmidt against the normal
lever arm	local z of the upper hit	constant sensorSeparation
precision	double	float

- **Both reduce to pC = d (x̂L·v)/(ẑL·v) when the two sensor centres are stacked along the normal and the planes are parallel.**
- **~570 µm is largest |pC| (flat-barrel sensor edge), about the size of the barrel-L1 width cut.**

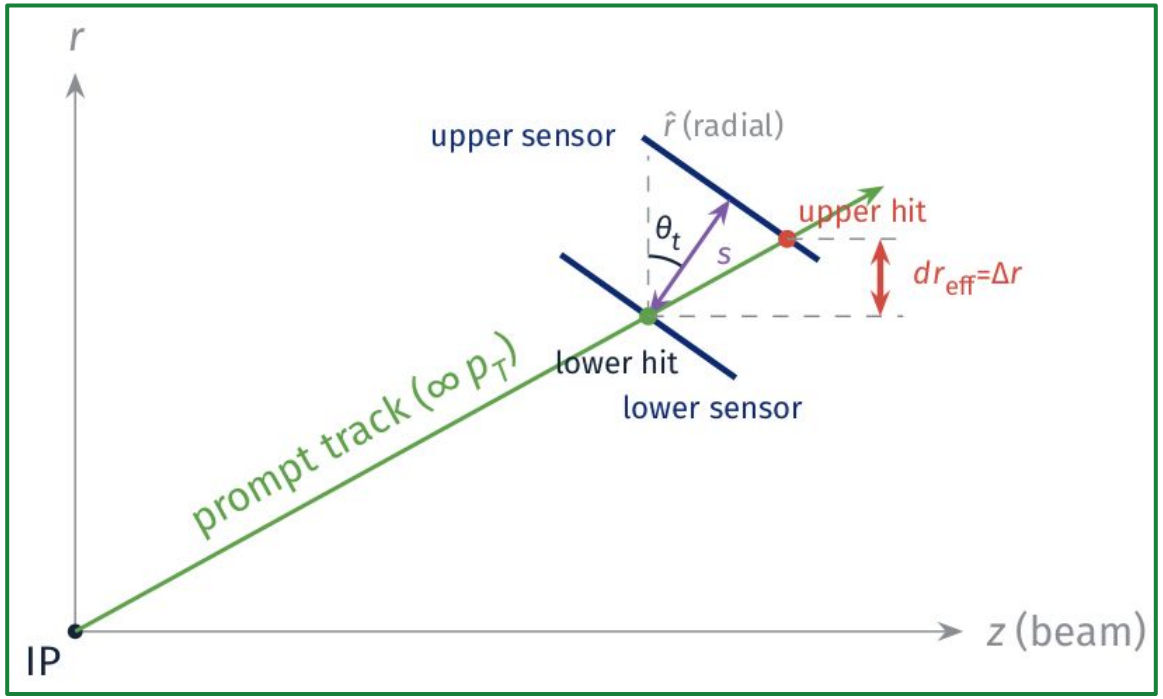
How dφ/dr is used in the CA Tracking

One number per stub, turned into a half-curvature the CA can compare between hits

- The stub stores a single direction summary, the azimuthal slope along the track:

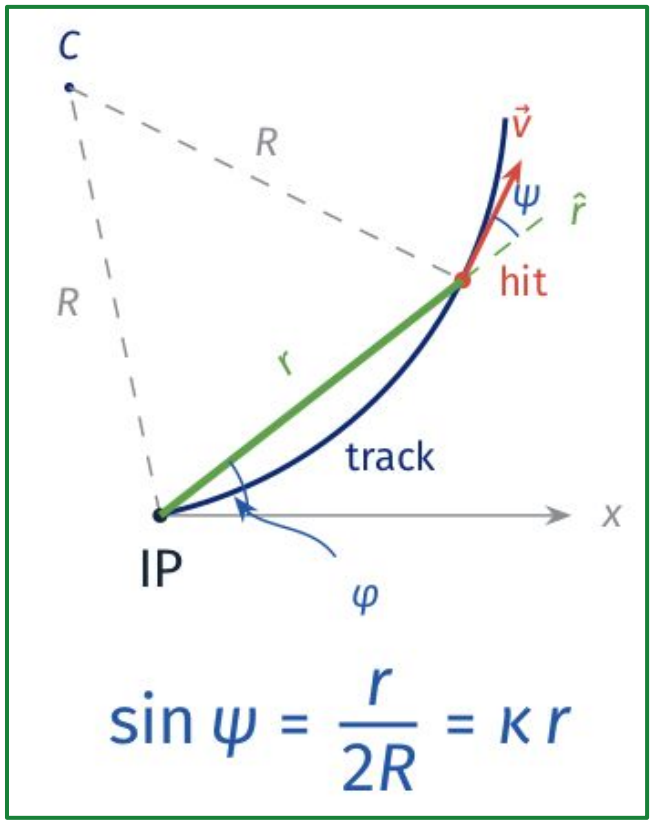
$$dPhiDr = \Delta\varphi_{corr} / dr_{eff}$$
$$dr_{eff} = s / (\cos\theta t + \sin\theta t \cdot z/r)$$

- dr_{eff} is the radial span a prompt track actually crosses between the two sensors: it folds in the module tilt θt and the slant z/r, so a central flat-barrel module gives dr_eff = s and the κ-cut means the same thing everywhere.
- σ(dPhiDr): each hit's global covariance propagated to φ, the two added in quadrature, divided by $|dr_{eff}|$.
- Nothing else is kept in the stub: no bend, no pT, no 4×4 covariance.



How dφ/dr is used in the CA Tracking

One number per stub, turned into a half-curvature the CA can compare between hits



- For a circle through the IP, the tangent-chord angle gives sin ψ = r/2R, while tan ψ = r · dPhiDr. Equating the two:

$$\kappa \equiv 1/(2R) = dPhiDr / \sqrt{(1 + r^2\, dPhiDr^2)}$$
$$\sigma\kappa = \sigma(dPhiDr) / (1 + r^2\, dPhiDr^2)^{(3/2)}$$

- Radius-independent: every stub on one track reports the same κ, so two stubs are directly comparable.
- Stub-stub doublet is killed if $|\kappa_i - \kappa_o| / \sqrt{(\sigma^2_{\kappa,i} + \sigma^2_{\kappa,o})} >$ **stubSigmaCut** (per layer pair, maxStubCurvSigma).
- Pixel-stub doublet: the doublet's own κd against the stub's κo, $|\kappa_d - \kappa_o| / \sigma_{\kappa,o}$; plus a sameDPhiSign requirement.